

Stanford CME 295 全课程讲义

Codex based on syllabus, slides, subtitles, and lecture videos

2026 年 4 月 13 日

目录

课程说明	2
课程目录	2
1 Lecture 1: Transformer	2
2 Lecture 2: Transformer-based models & tricks	14
3 Lecture 3: Large Language Models	27
4 Lecture 4: LLM training	42
5 Lecture 5: LLM tuning	54
6 Lecture 6: LLM reasoning	65
7 Lecture 7: Agentic LLMs	76
8 Lecture 8: LLM evaluation	87
9 Lecture 9: Current trends	99

课程说明

本讲义先逐讲生成，再按课程顺序合并。

课程目录

讲次	主题	日期
01	Lecture 1: Transformer	2025-09-26
02	Lecture 2: Transformer-based models & tricks	2025-10-03
03	Lecture 3: Large Language Models	2025-10-10
04	Lecture 4: LLM training	2025-10-17
05	Lecture 5: LLM tuning	2025-10-31
06	Lecture 6: LLM reasoning	2025-11-07
07	Lecture 7: Agentic LLMs	2025-11-14
08	Lecture 8: LLM evaluation	2025-11-21
09	Lecture 9: Current trends	2025-12-05

1 Lecture 1: Transformer

课程笔记

CME 295 课程笔记: Lecture 1

Transformer

Codex

2026-04-13



视频作者/频道: Stanford Online

发布日期: 2025-10-17

视频时长: 1:41:59

视频链接: <https://www.youtube.com/watch?v=Ub3GoFaUcds>

目录

1 课程导引与 NLP 任务图谱	2
1.1 本章小结	3
2 从分词到词向量：离散文本如何进入神经网络	3
2.1 分词的三种基本粒度	3
2.2 从 one-hot 到语义向量	4
2.3 本章小结	4
3 Transformer 之前：RNN、LSTM 与注意力的动机	4
3.1 RNN：顺序处理带来的上下文感知	5
3.2 LSTM：对长期依赖的工程修补	5
3.3 注意力为何成为转折点	6
3.4 本章小结	6
4 自注意力机制：让所有 token 直接相互作用	6
4.1 Query、Key、Value 的角色分工	6
4.2 注意力公式与含义	7
4.3 多头注意力的意义	8
4.4 本章小结	8
5 Transformer 架构：从输入到翻译输出的完整数据流	8
5.1 输入侧：token embedding 加位置编码	8
5.2 编码器与解码器各做什么	9
5.3 为什么需要 masked self-attention 与 cross-attention	9
5.4 计算技巧与训练细节	9
5.5 生成过程的闭环	10
5.6 本章小结	10
6 总结与延伸	10

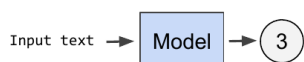
1 课程导引与 NLP 任务图谱

这节开篇课先回答两个问题：为什么要学 Transformer，以及它在自然语言处理（NLP）方法谱系中处在什么位置。讲者先说明课程目标，一是理解 Transformer 及其与大语言模型的关系，二是理解大模型如何训练、如何落到应用场景。对应的受众既包括准备把 LLM 作为职业方向的学生，也包括想做个人项目、或者仅仅希望建立 AI literacy 的学习者。

从课程定位看，这门课默认学生已经理解最基础的机器学习训练过程、神经网络和线性代数。这个前提很重要，因为后面所有模型都在处理两个核心对象：离散 token 与其向量表示，以及这些向量之间的相似性计算。讲者在开场反复强调，后面的公式并不是孤立数学，而是为了让模型学会“哪些词与哪些词更相关”。

NLP tasks overview

Classification



- Sentiment extraction
- Intent detection
- Language detection
- Topic modeling

Stanford University

ICME

图 1: 课程开篇用一张总览图把 NLP 任务分成分类、信息抽取、翻译等大类。

课件随后用情感分类、命名实体识别、机器翻译等任务说明 NLP 的输入输出形态并不相同：

- 分类任务是“文本进，标签出”，例如判断一句话的情感极性。
- 序列标注任务是“文本进，逐 token 标签出”，例如识别人名、地名、机构名。
- 生成任务是“文本进，文本出”，例如翻译、摘要、对话生成。

这一步并不是背景铺垫而已。它直接服务于后文的架构比较：有些模型适合拿整句表示做预测，有些模型擅长逐 token 条件生成，而 Transformer 之所以重要，是因为它后来演化出了同时覆盖这些任务形态的多种分支。

时间线的真正含义

课程中的时间线不是简单罗列年份，而是在说明 NLP 模型设计的焦点如何迁移：先从“如何按顺序处理文本”，发展到“如何学习更有语义的词向量”，再到“如何让任意位置的 token 直接交互”。

1.1 本章小结

第一部分的任务是建立全景视角：NLP 并不是一个单一任务，而是一组输入输出模式不同的问题；Transformer 也不是凭空出现，而是为了解决早期方法在表达能力、上下文建模和并行效率上的缺陷。

2 从分词到词向量：离散文本如何进入神经网络

模型不能直接处理原始字符串，因此第一步永远是 tokenization。讲者用 “A cute teddy bear is reading.” 这个例子说明，分词策略其实对应着不同的工程取舍。

2.1 分词的三种基本粒度

课程把分词方法分成三类：

- 词级分词：直观、可解释，但容易遇到 OOV (out-of-vocabulary) 问题，也很难共享词根信息。
- 子词分词：把高频词保留为整体，把低频词切成可复用片段，在词表规模与泛化能力之间折中。
- 字符级分词：几乎没有 OOV，但序列会显著变长，模型计算成本变高。

Tokenization summary

Method	Pros	Cons
Word-level	• Simple • Interpretable	• Risk of OOV • Does not leverage knowledge of root
Subword-level e.g. WordPiece, BPE	• Leverages common prefixes and suffixes • Learned from the data	• Risk of OOV, though less than word-level
Character-level	• Small chance of OOV • RoBUsT to CASinG anD MIspeliNGs	• Makes computations slower • Embeddings not interpretable

Stanford University

ICME

图 2: 课件用表格对比词级、子词级、字符级分词的优缺点。

讲者特别强调子词分词在现代模型中的关键性。因为语言里既存在常见完整词，也存在大量未登录词、专有名词、拼写变体。子词方法通过复用词根、前后缀和常见片段，让模型既能控制词表规模，又能在新词出现时拼装出可用表示。换句话说，分词并不只是预处理步骤，它直接决定了：

- 序列长度；
- 词表大小；
- OOV 风险；

- 训练和推理成本。

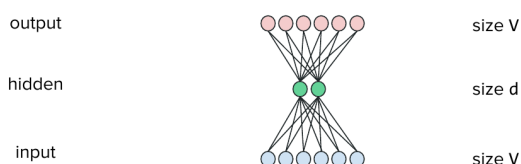
2.2 从 one-hot 到语义向量

如果把每个 token 表示成 one-hot 向量，那么不同 token 之间彼此正交，相似词不会更接近，模型也无法从几何结构里复用语义。因此课程引出 token representation，也就是 embedding。

embedding 的目标是把“相似 token 的向量相近、不相似 token 的向量较远”编码进连续空间。这样模型就能用线性层、相似度、聚类等方式直接利用语义结构。

Word2vec

Architecture



Stanford University

ICME

图 3: Word2vec 的基本结构：通过预测上下文或中心词来学习低维词向量。

以 Word2vec 为代表的早期词向量方法，其核心思想是“用上下文定义语义”。如果一个词经常出现在相似上下文中，它就应该得到相似表示。课程在这里并不深挖训练细节，而是强调了一个更重要的认识：embedding 不是人工定义的特征，而是通过训练目标从数据中学出来的表示空间。

静态词向量的局限

Word2vec 这类方法给同一个 token 分配固定向量，因此无法区分多义词在不同上下文中的含义。后续 RNN、注意力和 Transformer 的发展，本质上都在追求“上下文化表示”。

2.3 本章小结

文本进入模型之前必须经历两个步骤：先把字符串切成 token，再把 token 映射成向量。分词控制序列与词表的边界，embedding 则负责把离散符号变成可学习的几何对象。这为后面所有上下文建模方法奠定了输入表示基础。

3 Transformer 之前：RNN、LSTM 与注意力的动机

在 Transformer 之前，主流思路是按顺序读入 token。课程用 RNN 和 LSTM 回顾了这种范式的优势与瓶颈。

3.1 RNN: 顺序处理带来的上下文感知

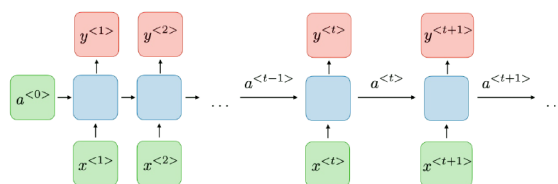
RNN 的优点在于它按时间顺序处理 token, 因此天然保留位置信息。当前时刻的隐藏状态由“当前输入 + 上一时刻状态”共同决定, 所以 token 的表示会受前文影响, 而不再是静态向量。

Recurrent Neural Networks (RNNs)

Overview

- First introduced in the 80s
- Class of neural networks where connections form a temporal sequence

General form



Stanford University Figure adapted from "VIP cheatsheets for Stanford's CS 230", Amidi, stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

ICME

图 4: RNN 通过递归隐藏状态顺序消费 token, 从而让词序进入表示。

这一步解决了词袋模型和静态 embedding 的一个核心缺陷: 词序终于进入模型了。可与此同时, 新的问题出现了:

- 序列必须逐步处理, 难以并行;
- 长程依赖会随着传播路径变长而变得难学;
- 早期 token 的信息容易在长序列中被冲淡。

3.2 LSTM: 对长期依赖的工程修补

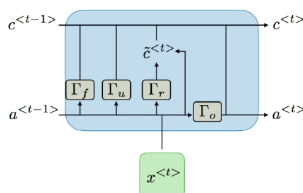
LSTM 通过更复杂的门控结构, 让模型在“保留什么、遗忘什么、输出什么”上有了更强控制能力。课程没有把重点放在门控方程推导, 而是强调它在历史位置上的作用: LSTM 是对 RNN 长期依赖问题的结构性补救。

Long Short-Term Memory (LSTM)

Overview

- Introduced in “Long short-term memory” (1997)
- Uses a more structured approach in the cell’s hidden state

General form



Stanford University Figure adapted from “VIP cheatsheets for Stanford’s CS 230”, Amidi, stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

ICME

图 5: LSTM 通过更结构化的内部状态和门控机制缓解长期依赖问题。

即便如此，LSTM 仍然保留了串行处理的本质。只要模型必须按 token 一个个向前推进，训练和推理就很难充分利用现代硬件的并行矩阵计算能力。

3.3 注意力为何成为转折点

课程随后引出 attention 的历史动机：在很多任务中，当前 token 并不只依赖最近一步，而是可能依赖很远位置的若干 token。与其把所有信息强行压缩进一个固定长度状态，不如直接让当前 token “看向” 整段上下文中最相关的位置。

这个视角非常关键。它意味着模型不再只能沿着时间链路传递信息，而是可以显式建立“我与谁更相关”的连接。Transformer 正是把这个想法推进到极致。

3.4 本章小结

RNN 与 LSTM 的历史作用在于：它们第一次让 token 表示真正依赖上下文，但代价是串行、难并行、长依赖难训练。注意力机制的出现，则把“记住全部过去”改写成“直接访问相关过去”，这就是 Transformer 的前奏。

4 自注意力机制：让所有 token 直接相互作用

讲者把自注意力描述为本讲的核心思想。对于一个 token，我们不再只看前一个隐藏状态，而是让它直接与序列中所有 token 交互，并根据相似度聚合信息。

4.1 Query、Key、Value 的角色分工

课程用一个直观化解释来讲 Q、K、V：

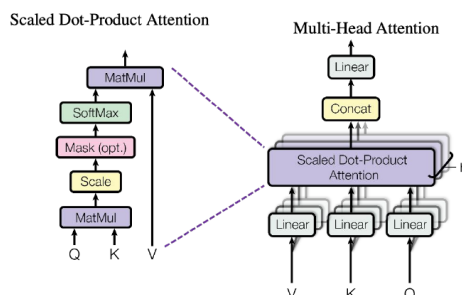
- Query 表示“我正在寻找什么信息”。
- Key 表示“我这里有什么可供匹配的信息”。
- Value 表示“一旦我被选中，我实际贡献什么内容”。

对某个 token 而言，它会把自己的 query 与所有 token 的 key 做比较，得到相关性权重，再对对应的 value 做加权求和，于是形成一个新的、上下文化的表示。

Attention mechanism

Efficient computations with matrices:

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

ICME

图 6: 自注意力把相似度计算写成统一的矩阵形式，从而适合硬件并行。

4.2 注意力公式与含义

课程给出的标准形式是：

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

式中各符号的含义如下：

- Q : 所有 query 组成的矩阵；
- K : 所有 key 组成的矩阵；
- V : 所有 value 组成的矩阵；
- QK^T : 两两 token 的相关性分数；
- d_k : key 的维度，用 $\sqrt{d_k}$ 缩放是为了稳定数值范围；
- softmax: 把分数变成权重分布；
- 最终结果: 对 value 的加权和，即新的上下文化表示。

这个公式最重要的不是记忆，而是理解它为什么高效。过去 RNN 风格方法依赖时间步递推；这里则把大量关系计算写成矩阵乘法和 softmax，因此可以在硬件上高度并行。

为什么说自注意力是“上下文化 embedding”？

因为输出不再只是 token 自己的静态表示，而是融合了整段序列中与它最相关的信息。一个 token 的新向量，取决于同一句话中其他 token 对它的贡献。

4.3 多头注意力的意义

课程最后补充了 multi-head attention 的直觉：不要只学习一种相似性，而是并行学习多种投影空间。不同 head 可能分别关注语法关系、指代关系、局部搭配或长程语义线索。训练过程中虽然没有硬性约束要求各 head 分工不同，但梯度下降往往会驱动它们学出互补表示。

4.4 本章小结

自注意力把序列建模从“按时间递推”改成“按相关性聚合”。Q/K/V 提供了统一接口，矩阵形式提供了并行能力，多头机制则让模型能够在多个关系子空间里同时建模。这是 Transformer 优于早期序列模型的第一根支柱。

5 Transformer 架构：从输入到翻译输出的完整数据流

当自注意力被组织进 encoder-decoder 框架时，Transformer 便成形了。课程这一部分最重要的价值在于：它不是只给出模块名称，而是用一个翻译示例把信息流串了起来。

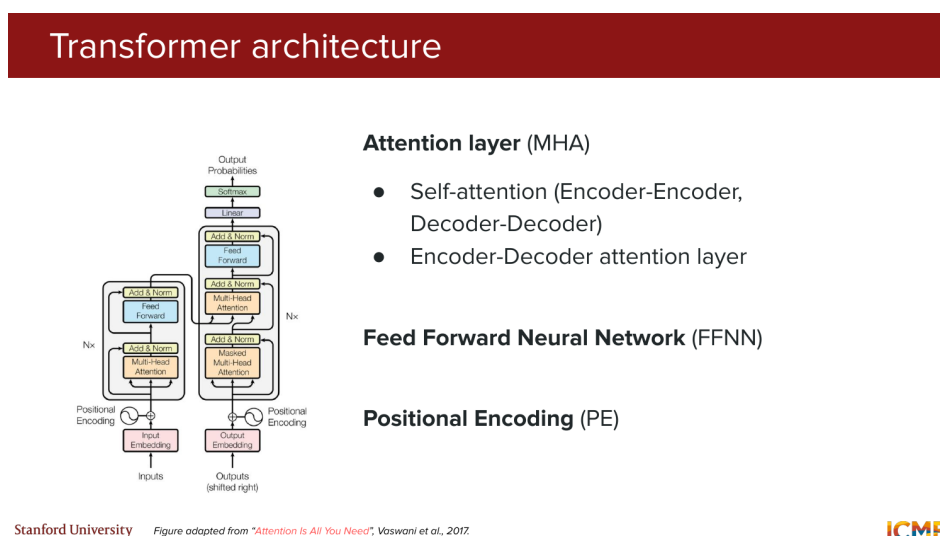


图 7: Transformer 的总架构：编码器负责输入表征，解码器负责条件生成。

5.1 输入侧：token embedding 加位置编码

由于自注意力本身不保留顺序，输入 token embedding 必须补上 positional encoding。课程在这一讲只给概念，不展开具体构造方法，但已经给出清晰定位：位置编码不是额外点缀，而是让模型分辨顺序的必要信号。

编码器侧的数据流可以概括为：

1. 文本被切成 token；
2. token 被映射成 embedding；
3. 加入位置编码形成 position-aware embeddings；
4. 送入多层 encoder block。

5.2 编码器与解码器各做什么

每个 encoder block 主要包含两类子层：

- 多头自注意力层：让输入序列内部充分交互；
- 前馈网络（FFN）：对每个位置的表示做更强的非线性变换。

解码器则更复杂一些，它既要看已经生成的输出前缀，又要看编码器产出的输入表示，因此包含：

- masked self-attention：只能看当前位置左侧，保证因果性；
- cross-attention：把解码器当前状态与编码器输出对齐；
- FFN：进一步丰富表示。

5.3 为什么需要 masked self-attention 与 cross-attention

masked self-attention 的目的是防止信息泄漏。做下一个 token 预测时，模型不能偷看未来 token，因此解码器每个位置只能访问自己和之前位置。

cross-attention 则承担“条件生成”的角色。解码器当前状态提出 query，编码器输出提供 key 和 value，模型据此决定源句中哪些位置最有助于当前目标词生成。机器翻译正是这种结构最典型的应用：左边编码英文句子，右边逐词解码法文句子。

Stitching all the pieces together with an example

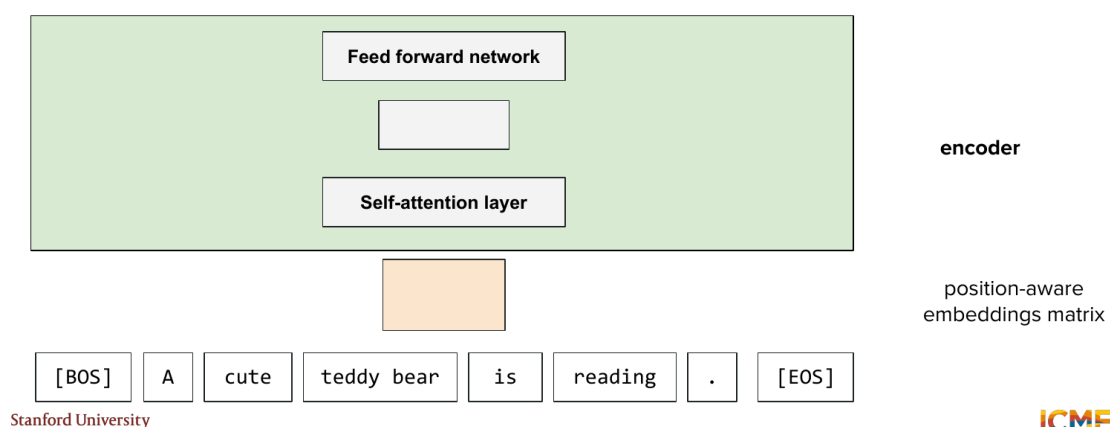


图 8: 课件通过逐步拼装示例，把 embedding、自注意力、FFN 和 encoder 串成一条可追踪的数据流。

5.4 计算技巧与训练细节

课程还点到两个原始 Transformer 论文中的重要技巧：

- Multi-head attention：让多组投影并行工作，增强表达能力；

- Label smoothing: 在训练时避免目标分布过于尖锐, 改善泛化和校准。

另外, FFN 的隐藏层通常比输入输出维度更大。讲者在问答里明确说明, 这样做是为了给模型更高的特征变换自由度, 而不是像 Word2vec 那样做压缩瓶颈。

5.5 生成过程的闭环

解码从 [BOS] 开始, 模型在 softmax 层上得到整个词表的概率分布, 选出下一个 token 后再送回解码器继续迭代。当生成到 [EOS] 时, 序列结束。这个循环就是后续所有自回归语言模型推理的基础范式。

5.6 本章小结

Transformer 可以看成把“位置编码 + 自注意力 + FFN + 因果掩码 + 编解码交互”系统性拼接在一起。编码器负责把输入压成上下文化表示, 解码器负责在条件约束下逐 token 生成输出, 而这套流程后来成为现代大语言模型演化的出发点。

6 总结与延伸

这节课的真正任务不是把 Transformer 一次性讲到最细, 而是搭建一条完整的认知路径:

1. 文本必须先被切成 token, 并映射为向量;
2. 静态 embedding 不够, 因为语义依赖上下文;
3. RNN/LSTM 虽然能建模顺序, 但串行且难处理长程依赖;
4. 注意力机制允许 token 直接访问相关上下文;
5. Transformer 把注意力机制工程化、模块化, 并借助矩阵并行获得巨大可扩展性。

如果把本讲看成整门课的地基, 那么最值得带走的不是若干缩写, 而是三条主线:

- **表示主线**: 从离散 token 到上下文化向量表示;
- **结构主线**: 从顺序递推到全局交互;
- **计算主线**: 从难并行的时间链路到适合硬件的矩阵运算。

后续课程将在这个基础上继续展开: 第二讲会讨论位置编码、归一化和模型变体, 第三讲则会真正进入大语言模型本体, 包括解码策略、提示学习和推理优化。回看本讲, Transformer 之所以成为主角, 并不是因为名字响亮, 而是因为它第一次把“表达能力”“全局依赖建模”和“可扩展计算”同时放进了同一套架构里。

2 Lecture 2: Transformer-based models & tricks

课程笔记

CME 295 课程笔记: Lecture 2

Transformer-Based Models & Tricks

Codex

2026-04-13



视频作者/频道: Stanford Online

发布日期: 2025-10-17

视频时长: 1:47:19

视频链接: <https://www.youtube.com/watch?v=yT84Y5zCnaA>

目录

1 从原始 Transformer 到现代变体：本讲的目标	2
1.1 本章小结	2
2 位置编码的演进：从绝对位置到相对位置信息	2
2.1 为什么必须注入位置信息	2
2.2 正弦位置编码的公式与动机	2
2.3 从绝对位置到相对位置：ALiBi 与 RoPE	3
2.4 本章小结	4
3 层归一化与训练稳定性：Post-Norm 到 Pre-Norm	4
3.1 本章小结	5
4 注意力近似与结构省算：局部窗口、MQA 与 GQA	5
4.1 全局自注意力的代价	5
4.2 局部与全局注意力的折中	5
4.3 从 MHA 到 MQA/GQA	6
4.4 本章小结	7
5 Transformer-based 模型家族与 BERT 深入解析	7
5.1 三大模型家族	7
5.2 为什么是 BERT	8
5.3 BERT 的预训练任务	8
5.4 从预训练到微调	9
5.5 本章小结	9
6 BERT 的局限与两类变体：DistilBERT、RoBERTa	9
6.1 BERT 的局限	9
6.2 DistilBERT：更小更快	10
6.3 RoBERTa：把预训练做得更扎实	10
6.4 本章小结	10
7 总结与延伸	10

1 从原始 Transformer 到现代变体：本讲的目标

第二讲不是重新介绍 Transformer，而是回答两个更工程化的问题：原始 Transformer 中哪些部件后来被改掉了，以及为什么要改。课程先用上一讲的 attention map 和架构图做 recap，再明确本讲的主题包括位置编码、层归一化、注意力近似、Transformer-based 模型家族，以及 BERT 的深入分析。

这一讲的总体逻辑非常清楚：Transformer 的主干思想几乎没变，但一些局部机制会显著影响长度外推能力、训练稳定性、推理内存占用和任务适配方式。因此，理解“tricks”不是记零散技巧，而是理解现代模型为何在原论文基础上继续演化。

1.1 本章小结

第二讲的定位是“结构细化课”。它默认你已经理解自注意力与 encoder-decoder 主干，现在开始讨论哪些设计在实践里更稳、更省、更适合现代大模型。

2 位置编码的演进：从绝对位置到相对位置信息

2.1 为什么必须注入位置信息

自注意力允许任意 token 直接交互，但这也意味着如果不额外处理，模型只看到一个集合，而看不到顺序。课程把这个问题表述为：direct links 会“丢掉”位置信息。所以原始 Transformer 需要把 position embedding 加到 token embedding 上。

最早的做法是每个位置学一个独立向量，然后直接相加。优点是简单；缺点是如果训练时只见过长度 L 的序列，那么位置 $L + 1$ 之后就没有学习到的 embedding，长度外推会失败。

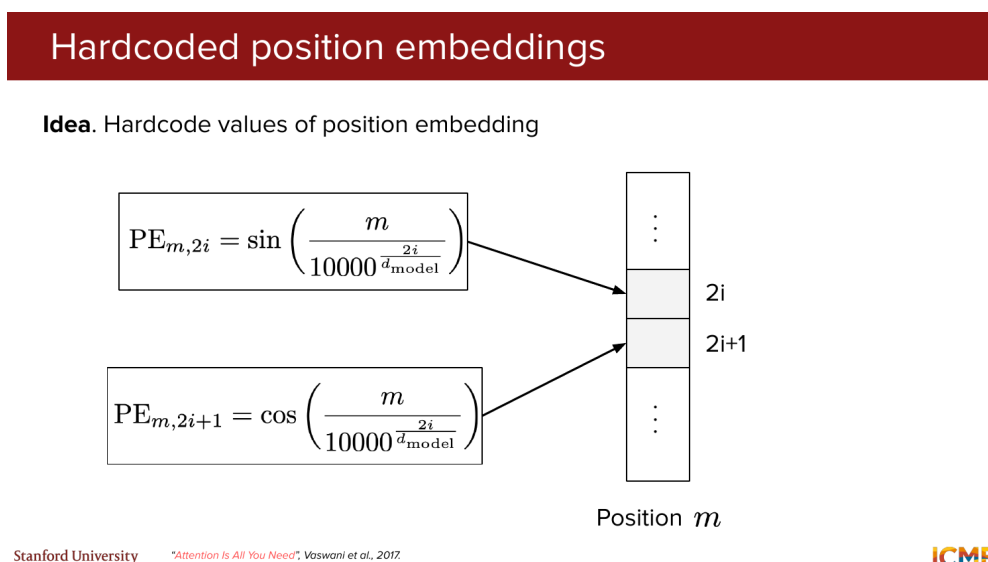


图 1: 原始 Transformer 用正弦和余弦构造硬编码位置向量。

2.2 正弦位置编码的公式与动机

课程随后转向 hardcoded position embeddings，也就是经典的正弦位置编码：

$$\text{PE}_{(m,2i)} = \sin\left(\frac{m}{10000^{2i/d}}\right), \quad \text{PE}_{(m,2i+1)} = \cos\left(\frac{m}{10000^{2i/d}}\right)$$

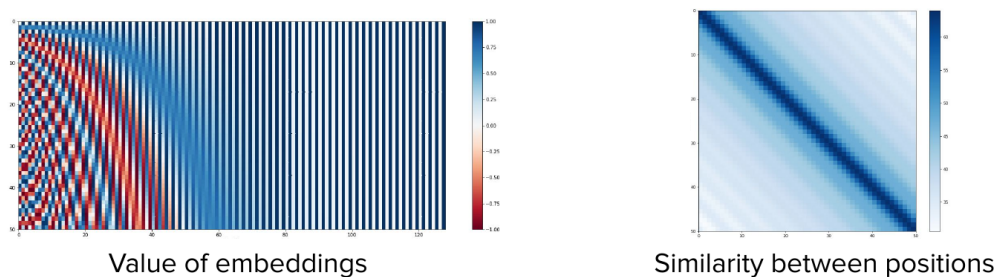
符号说明如下：

- m : 序列中的位置索引；
- i : embedding 维度中的第 i 个频率块；
- d : 位置向量总维度；
- 偶数维用正弦，奇数维用余弦；
- 不同频率让模型同时编码近距离和远距离的位置信号。

这类编码的关键优势不是“更优雅”，而是可以无须为每个位置单独学习参数，因此天然支持更长序列的外推。课程还特别强调，相似位置在这种表示下会有可解释的几何关系。

Hardcoded position embeddings

Idea. Hardcode values of position embedding



Stanford University "Attention Is All You Need", Vaswani et al., 2017, "Transformer Architecture: The Positional Encoding", by Kazemnejad, 2019.

ICME

图 2: 正弦位置编码既给出随位置变化的波形，也诱导出位置之间的相似性结构。

2.3 从绝对位置到相对位置：ALiBi 与 RoPE

讲者随后指出，注意力真正关心的往往不是“绝对第几位”，而是“彼此相距多远”。这就引出相对位置思想：不要只在输入端相加位置向量，而要直接在 attention 打分里引入位置信息。

ALiBi 的做法是在 query-key 分数中加入线性 bias，使距离更远的 token 在没有特殊证据时自然受到更强惩罚。它参数更少，也有不错的长度外推表现。

如今更常见的选择是 RoPE (Rotary Position Embeddings)。RoPE 的思想不是额外加一个位置向量，而是对 query 与 key 做旋转变换。二维情形下可以写成：

$$R_m = \begin{bmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{bmatrix}$$

其中：

- m : 位置；
- θ : 不同维度块使用的旋转频率；

- $R_m q$ 与 $R_n k$ 的内积会自然带出 $m - n$ 的相对位置信息。

Default choice nowadays: rotations in attention layer

Idea. Rotate query and key vectors with rotation matrix



Stanford University "RoFormer: Enhanced Transformer with Rotary Position Embeddings", Su et al., 2021.
Figure from "Super Study Guide: Transformers & Large Language Models", Amidi et al., 2024.

ICME

图 3: RoPE 把位置信息写进 query/key 的旋转变换，而不是只加到输入 embedding 上。

为什么 RoPE 在现代模型里常见

因为它把相对位置信息直接注入相似度计算本身，既贴合 attention 的工作位置，又兼顾外推能力和工程简洁度。课程也指出今天大量模型都使用 RoPE 或其变体。

2.4 本章小结

位置编码的演进可以概括为三步：先有可学习绝对位置向量，再有可外推的正弦位置编码，最后进一步发展到直接作用于注意力分数或 query/key 表示的相对位置方法。今天主流选择偏向 RoPE，本质上是因为它更贴近注意力机制本身。

3 层归一化与训练稳定性：Post-Norm 到 Pre-Norm

课程提醒我们，Transformer 之所以能堆很多层，不只靠注意力本身，也靠归一化和残差连接控制训练动态。Layer Normalization (LN) 是在单个样本内部、沿特征维做标准化：

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

其中：

- x : 某个位置的隐藏向量；
- μ, σ^2 : 该向量各维度上的均值与方差；
- ϵ : 防止分母为零的稳定项；
- γ, β : 可学习的缩放和平移参数。

Different kinds of layer normalization

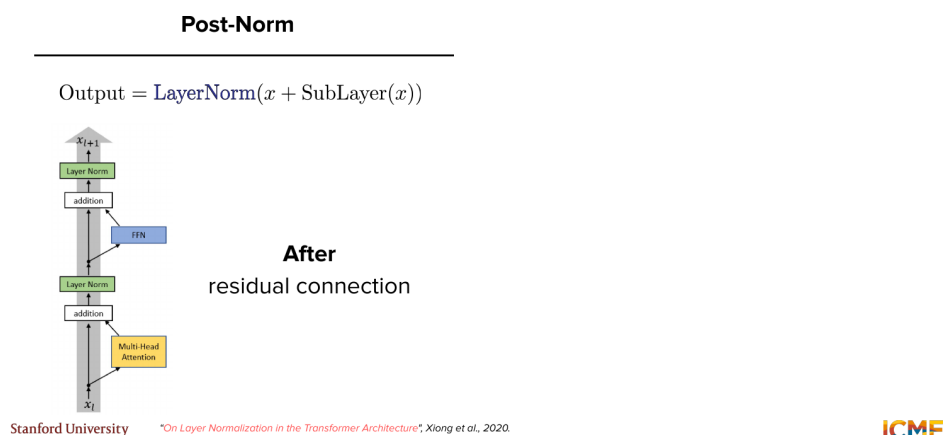


图 4: 课程用结构图比较 LayerNorm 放在子层前后带来的差异。

原始 Transformer 采用 Post-Norm, 也就是先做子层计算与残差相加, 再归一化。后来很多模型改成 Pre-Norm, 即先归一化, 再进入 attention 或 FFN 子层。讲者给出的核心判断是: Pre-Norm 往往更利于深层网络的梯度传播, 因此“如今”更常见。

从理解上说, 归一化并不是附属模块, 它决定了深层堆叠是否稳定。只要模型层数、参数量和训练语料继续上升, 这类稳定性设计就会持续重要。

3.1 本章小结

LayerNorm 的公式不复杂, 但位置放置会影响整个模型的优化路径。现代大模型偏向 Pre-Norm, 不是因为原论文错了, 而是因为规模继续扩大后, 训练稳定性成为首要工程约束。

4 注意力近似与结构省算: 局部窗口、MQA 与 GQA

4.1 全局自注意力的代价

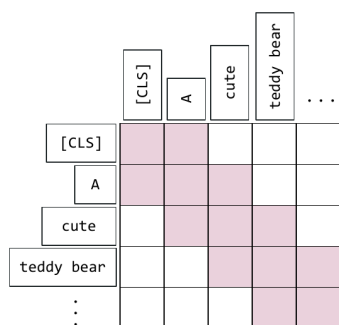
如果每个 token 都和所有 token 交互, 那么长度为 n 的序列需要计算 $n \times n$ 的相关性矩阵。随着 context 增大, 显存和计算都会迅速上涨。因此现代模型不再总是坚持“所有位置全连”。

4.2 局部与全局注意力的折中

课程先介绍 sliding window attention (SWA): 每个 token 只看局部邻域, 而不是整段序列。它的思想与卷积类似, 利用语言中很多依赖是局部的事实, 把昂贵的全局关系计算近似成局部窗口运算。

Leveraging local and global attention

SWA = Sliding Window Attention



Variations include **interleaving local** and **global attention layers**

Stanford University

ICME

图 5: SWA 只保留局部邻域的注意力连接，用更低成本保留主要局部依赖。

课程也提到可以交替使用 local 与 global attention 层。换句话说，局部近似不是否认长程依赖，而是把“每一层都全局交互”改成“有计划地稀疏化交互”。

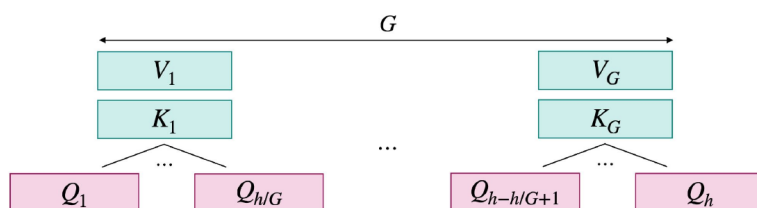
4.3 从 MHA 到 MQA/GQA

另一个省算方向出现在 head 的 key/value 组织方式上。标准 multi-head attention 中，每个 query head 都有自己独立的 key 和 value 投影，这会推高 KV cache 和推理内存。

MQA (Multi-Query Attention) 让多个 query head 共享同一组 key/value; GQA (Grouped-Query Attention) 则在两者之间折中，让若干 query head 共享一组 key/value。这样做会牺牲一部分自由度，但可以显著降低推理内存压力。

Sharing attention heads

Idea. Share key/value attention heads within groups of queries



Stanford University

Figure from "Super Study Guide: Transformers & Large Language Models", Amidi et al., 2024.

ICME

图 6: 课程用共享示意图说明 MQA/GQA 如何减少 key/value 的冗余存储。

这一节的统一视角

不论是 SWA 还是 MQA/GQA，它们都不是在改变 Transformer 的基本语义，而是在回答同一个工程问题：当序列更长、模型更大时，如何让注意力的表达能力和推理成本保持平衡。

4.4 本章小结

现代 Transformer 的 “tricks” 很多都围绕注意力成本展开：要么减少可见连接数，要么压缩 key/value 的复制开销。理解这一点后，后面看到长上下文模型和高吞吐服务框架时，就不会把它们误认为完全不同的架构。

5 Transformer-based 模型家族与 BERT 深入解析

5.1 三大模型家族

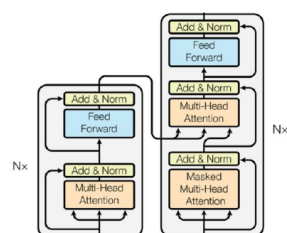
课程先回到高层分类。常见 Transformer-based 模型主要有三类：

- encoder-decoder
- encoder-only
- decoder-only

Overview of Transformer-based models

3 categories of Transformer-based models:

Encoder-decoder	Text to text	T5, mT5, ByT5
------------------------	--------------	---------------



Stanford University Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

ICME

图 7: 三类 Transformer-based 模型的结构与典型任务：编码-解码、纯编码、纯解码。

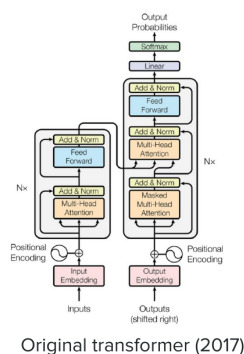
这三类结构并不是谁替代谁，而是分别对应不同任务偏好：

- encoder-decoder 更适合 text-to-text 变换；
- encoder-only 更适合分类、检索、抽取等“先编码再判别”的任务；
- decoder-only 更适合自回归生成。

5.2 为什么是 BERT

BERT 是 Bidirectional Encoder Representations from Transformers。它保留原始 Transformer 的 encoder 部分, 去掉 decoder, 从而得到一个能双向聚合上下文的编码模型。课程专门强调了“bidirectional”这一点: 它不是只看左侧上下文, 而是能同时利用左右信息。

BERT architecture



Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

ICME

图 8: BERT 继承原始 Transformer 的 encoder 堆栈, 把重点放在输入表示学习而非文本生成。

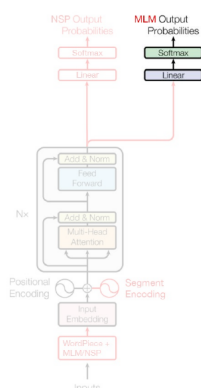
在输入构造上, BERT 不只是 token embedding, 还会加入 position embedding 和 segment embedding, 用来区分句对输入中的不同句子。特殊 token 如 [CLS] 和 [SEP] 也在后续任务适配中扮演关键角色。

5.3 BERT 的预训练任务

课程讲了 BERT 的两个经典 proxy task:

- MLM (Masked Language Modeling): 随机挑出部分 token 做预测;
- NSP (Next Sentence Prediction): 判断两个句子是否相邻。

Proxy task: Masked Language Modeling



Idea. 15% of input tokens are set up for prediction where:

- 80% are masked
- 10% are changed to a random word
- 10% are unchanged

Benefits

- Network learns language modeling based on contextual information
- Regularization reflects probabilistic nature of language

Stanford University Figure adapted from "Attention Is All You Need", Vaswani et al., 2017.

ICME

图 9: MLM 通过掩码预测学习双向上下文表征, 是 BERT 预训练的核心任务。

其中 MLM 是最本质的一项。课程明确提到典型设置是 15% token 参与预测，其中大部分替换为 [MASK]，少部分替换成随机词或保持不变。这么做的目的是避免模型只会识别 [MASK] 这个特殊训练场景。

5.4 从预训练到微调

BERT 的强项在于它学到的 contextual embeddings 可以直接迁移到下游任务。做情感分类时，可以把 [CLS] 的最终表示接一个小型分类头；做问答时，则可以在每个位置上预测答案起点与终点。

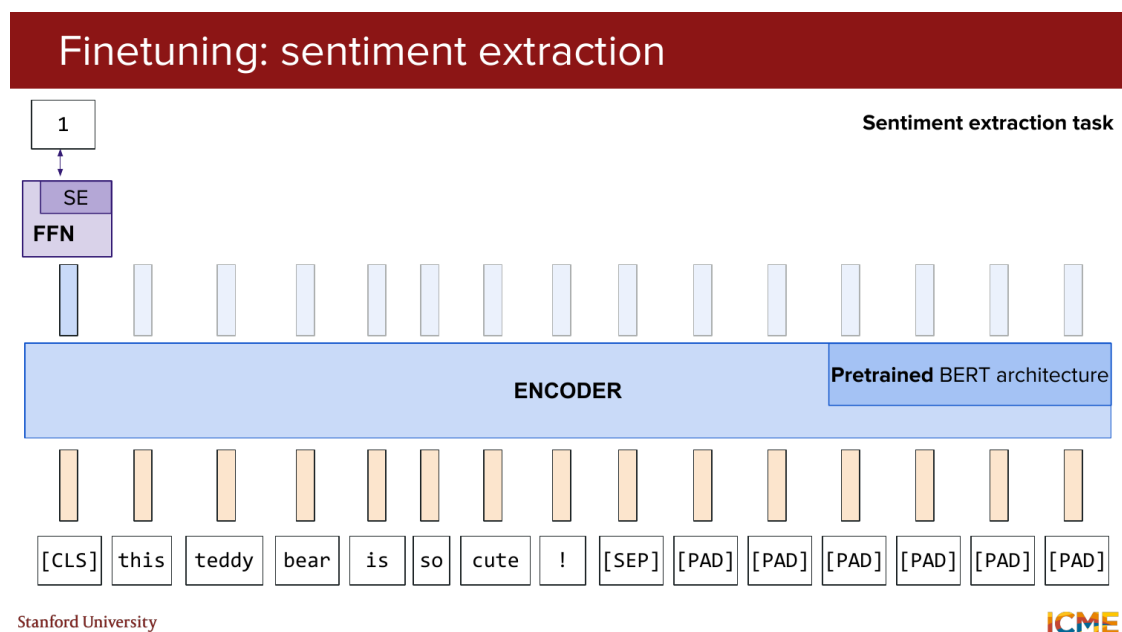


图 10: BERT 微调示意：把预训练 encoder 产生的上下文文化表示接到具体任务头上。

这也是 BERT 曾经如此成功的原因：它把“大量无监督语料上的表示学习”和“小数据监督任务上的快速适配”分离开来，大幅提高了 NLP 任务的迁移效率。

5.5 本章小结

BERT 的意义不只是一个模型名，而是一个范式：先用大语料学强表征，再用简单任务头做下游适配。它代表了 encoder-only 路线的成熟形态，也为后续各种判别式文本模型打下了基础。

6 BERT 的局限与两类变体：DistilBERT、RoBERTa

6.1 BERT 的局限

课程总结了 BERT 的几个现实限制：

- 上下文长度有限，原始版本通常只到 512；
- 参数量和延迟仍然偏高；
- 预训练目标是否最优并不确定，尤其 NSP 的必要性值得怀疑。

这些问题催生了两条不同方向的改进：一条是更省，一条是更强。

6.2 DistilBERT: 更小更快

DistilBERT 的核心思路是蒸馏。教师模型输出的不只是硬标签，而是一整条概率分布；这条分布比单点标签包含更多关于类别相似性和不确定性的知识。学生模型通过拟合教师分布，在更小参数规模下尽量保留性能。

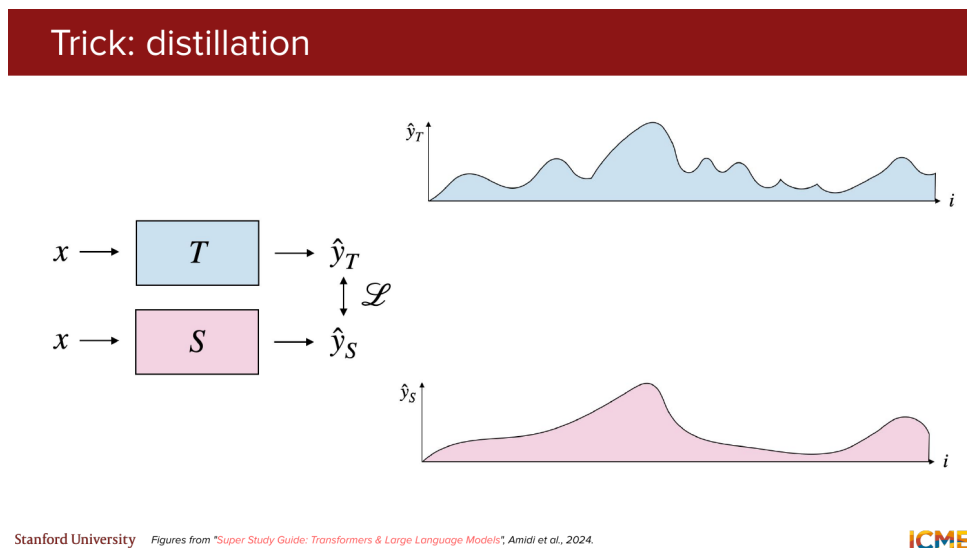


图 11: 蒸馏把教师模型的软分布迁移给学生模型，以更少层数换取更快推理。

课程在口头解释里还补充了一个很重要的判断：DistilBERT 之所以高效，不只是因为层数减半，更因为它把“老师学到的结构性概率信息”也迁移了过去。

6.3 RoBERTa: 把预训练做得更扎实

RoBERTa 并没有发明全新结构，而是系统地审视 BERT 训练流程：如果拿掉 NSP、改成动态 masking、增大数据量、训练更久，会发生什么？课程给出的结论是，这些训练层面的改良本身就能显著提升性能。

这一结论的启发

很多模型改进并不来自结构创新，而来自训练设定的重新审视。RoBERTa 说明“把已有架构训练对”本身就是一种重要研究方向。

6.4 本章小结

DistilBERT 和 RoBERTa 分别代表了两种不同优化目标：前者是在性能可接受前提下更省资源，后者是在同类结构上把预训练流程打磨到更高上限。它们说明 BERT 之后的进展，不只是“换架构”，也包括“换训练策略”和“换部署成本曲线”。

7 总结与延伸

第二讲把第一讲的骨架真正拉成了现代模型设计图。最值得提炼的认识有四点：

1. **位置信息不是附件，而是自注意力成立的前提。**从 learned PE 到 sinusoidal PE，再到 ALiBi 和 RoPE，变化的是注入方式，不变的是“必须显式处理顺序”。
2. **模型能否稳定堆深，取决于归一化与残差布局。** LayerNorm 的放置位置是训练工程的重要决策。
3. **注意力的成本决定了现代架构细节。**局部窗口、MQA、GQA 都是在为长上下文和低内存推理让路。
4. **BERT 展示了 encoder-only 路线的威力。**通过预训练表示再微调任务头，NLP 进入了大规模迁移学习时代。

如果第一讲回答的是“Transformer 是什么”，那么第二讲回答的是“为什么今天的模型长得不完全像 2017 年论文里的样子”。现代大模型并没有背离 Transformer，恰恰相反，它们是在保留主干思想的前提下，不断修补最影响长度、稳定性和成本的局部机制。理解这些变化，后面再看 LLM、KV cache、长上下文和推理优化时，就能把它们放回同一条技术脉络之中。

3 Lecture 3: Large Language Models

课程笔记

CME 295 课程笔记: Lecture 3 Large Language Models

Codex

2026-04-13



视频作者/频道: Stanford Online

发布日期: 2025-10-17

视频时长: 1:48:45

视频链接: <https://www.youtube.com/watch?v=Q5baLehv5So>

目录

1 什么是 LLM：定义、尺度与主干结构	2
1.1 本章小结	2
2 MoE：为什么不必每次都激活全部参数	2
2.1 从“所有参数一起算”到“只选部分专家”	2
2.2 Dense MoE 与 Sparse MoE	3
2.3 MoE 放在 Transformer 的哪里	3
2.4 路由塌缩与负载均衡	3
2.5 本章小结	4
3 从 next-token prediction 到具体解码策略	4
3.1 语言模型真正学到的是什么	4
3.2 Greedy decoding 与 Beam search	5
3.3 Sampling、Top-k 与 Top-p	6
3.4 Temperature 与 guided decoding	6
3.5 本章小结	7
4 上下文窗口与提示策略：从 ICL 到 CoT	8
4.1 Context length 不只是一个数字	8
4.2 In-Context Learning	8
4.3 Chain of Thought 与 Self-Consistency	8
4.4 本章小结	9
5 推理优化：KV cache、PagedAttention 与更快解码	9
5.1 为什么推理会慢	9
5.2 进一步压缩 KV 的存储成本	10
5.3 Speculative decoding 与 Multi-Token Prediction	11
5.4 本章小结	12
6 总结与延伸	13

1 什么是 LLM：定义、尺度与主干结构

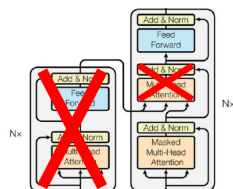
第三讲正式进入大语言模型本体。课程先用上一讲的三类 Transformer-based 模型做 recap，然后把范围收缩到今天最主流的一类：decoder-only、自回归、text-to-text 的大规模语言模型。

讲者对“large”的定义非常工程化，不是模糊形容词，而是至少包含三层含义：

- 参数规模大，通常从十亿级往上走；
- 训练数据量大，常见为数百亿到万亿 token；
- 训练和推理所需算力大，需要大量 GPU 资源和复杂系统优化。

Characteristics

Decoder-only Transformer-based model



Stanford University Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

ICME

图 1: 课程强调现代 LLM 的主干几乎都是 decoder-only Transformer。

课程还特别澄清了一个常见误解：不是所有“大模型”都算今天语境里的 LLM。按照本讲的定义，BERT 这类 encoder-only 模型虽然参数不小，但不做自回归文本生成，因此通常不被归入现代 LLM 的主流范畴。

本讲默认的 LLM 定义

一个现代 LLM 通常是 decoder-only Transformer，通过 next-token prediction 训练，在超大语料和超大算力下学到能够做文本生成的条件分布模型。

1.1 本章小结

LLM 首先是 language model，其次才是 large。课程之所以从结构、数据和算力三个维度同时定义它，是为了说明后面所有技巧都围绕这三个量级约束展开。

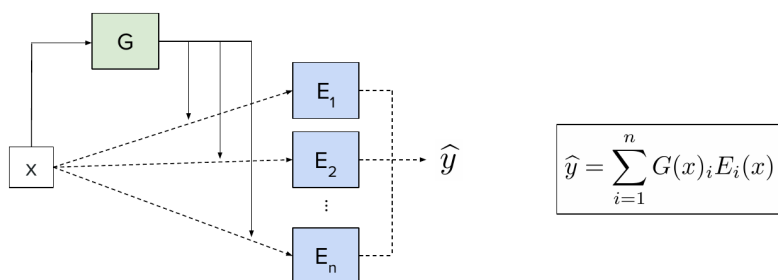
2 MoE：为什么不必每次都激活全部参数

2.1 从“所有参数一起算”到“只选部分专家”

如果一个模型有海量参数，那么每次前向都让所有参数参与计算，会带来极高成本。课程用“房间里有数学家、物理学家、化学家和历史学家，你问一个数学问题是否需要所有人一起回答”这个比喻引出 Mixture of Experts (MoE)。

Overview of MoEs

MoE = Mixture of Experts



Stanford University

ICME

图 2: MoE 的核心结构: router/gate 根据输入选择少量专家参与计算。

在 MoE 中, 给定输入 x , 会有一个 gate 或 router 决定哪些 expert 需要被激活。于是模型的“总参数量”可以很大, 但“单次激活参数量”相对较小, 从而提高参数规模与推理成本之间的比例。

2.2 Dense MoE 与 Sparse MoE

课程区分了两类 MoE:

- Dense MoE: 所有 expert 都参与, 只是权重不同;
- Sparse MoE: 只选极少数 expert, 真正做到稀疏激活。

现代大模型讨论的 MoE, 主要指 sparse MoE。它的价值就在于: 总参数可以继续扩张, 但每个 token 实际只路由到少数专家, 不必让全部参数同时上线。

2.3 MoE 放在 Transformer 的哪里

课程说明, MoE 通常不是替换 attention, 而是替换或增强 FFN 子层。理由很直观: attention 负责 token 间的信息交互, 而 FFN 更像逐位置的特征变换模块, 把 expert 机制放在这里更自然。

2.4 路由塌缩与负载均衡

MoE 的难点不在概念, 而在训练。如果 router 总是把大多数 token 分给同一个 expert, 就会出现 routing collapse, 导致专家利用率极不平衡、学习效率很差。

Training challenges include routing collapse

Symptom. Same expert gets selected most of the time.

"routing collapse"

Stanford University "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity", Fedus et al., 2021.

ICME

图 3: 课程把 routing collapse 定义为大多数 token 总是流向同一专家。

因此训练时通常要加入负载均衡目标, 让不同 expert 获得更均匀的 token 流量。课程在口头说明里也强调, MoE 路由通常是在 token 级别发生的, 即同一序列中的不同 token 完全可能流向不同专家。

Interpreting experts

Layer 0

```
class MoELayer(nn.Module):
    def __init__(self, experts: List[nn.Module]):
        super().__init__()
        assert len(experts) > 0
        self.experts = nn.ModuleList(experts)
        self.gate = gate
        self.args = moe_args

    def forward(self, inputs: torch.Tensor):
        inputs_squashed = inputs.view(-1, inputs.size(-1))
        gate_logits = self.gate(inputs_squashed)
        weights, selected_experts = torch.topk(
            gate_logits, self.args.num_experts_per_token
        )
        weights = nn.functional.softmax(
            weights,
            dim=-1,
            dtype=torch.float,
        ).type_as(inputs)
        results = torch.zeros_like(inputs_squashed)
        for i, expert in enumerate(self.experts):
            batch_idx, nth_expert = torch.where(
                results == weights[batch_idx]
            )
            results_squashed = expert(inputs_squashed[batch_idx])
            results.view_as(inputs)
        return results.view_as(inputs)
```

Each color represent an expert

Stanford University "Mixture of Experts", Jiang et al., 2024.

ICME

图 4: 按 token 着色的专家路由图说明, 不同 token 会被送往不同 expert。

2.5 本章小结

MoE 的核心不是“多几个模块”, 而是把模型扩张方式从“所有参数永远一起工作”改成“有条件地激活部分参数”。它让参数规模继续上升成为可能, 但代价是必须解决路由训练、负载均衡和系统实现问题。

3 从 next-token prediction 到具体解码策略

3.1 语言模型真正学到的是什么

课程从最基本的 next-token prediction 出发, 强调语言模型学习的是条件分布:

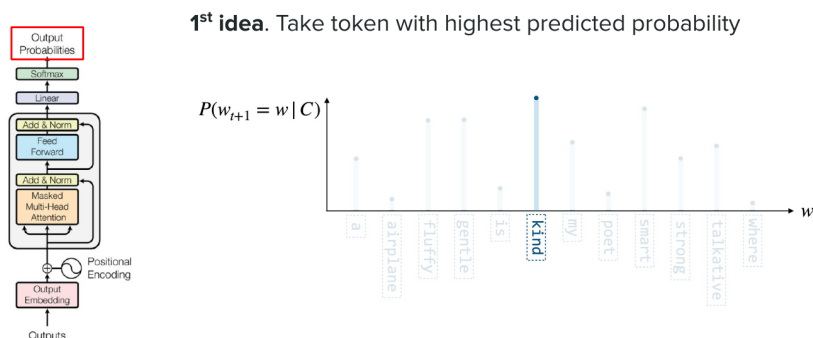
$$p_{\theta}(x_t | x_{<t})$$

其中:

- $x_{<t}$: 当前位置之前的 token 前缀;
- x_t : 当前位置候选 token;
- p_θ : 模型参数为 θ 的条件概率分布。

训练时模型学的是这个分布; 推理时真正困难的, 是如何从这个分布中选 token。课程用 greedy、beam search 与 sampling 三种策略逐步展开。

Predicting next token with greedy decoding



Stanford University

Figure adapted from "Attention is All You Need", Vaswani et al., 2017. "Super Study Guide: Transformers and Large Language Models", Amidi et al., 2024.

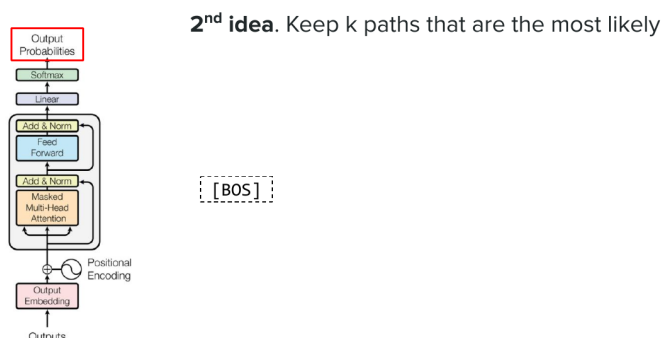
ICME

图 5: Greedy decoding 每一步都选当前概率最高的 token。

3.2 Greedy decoding 与 Beam search

Greedy decoding 最简单, 但只追求局部最优, 容易在整体序列层面不自然或不够多样。Beam search 则保留多个候选路径, 尝试避免局部最优陷阱。

Predicting next token with beam search



Stanford University

Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

ICME

图 6: Beam search 通过维护多个候选前缀, 寻找更高整体概率的序列。

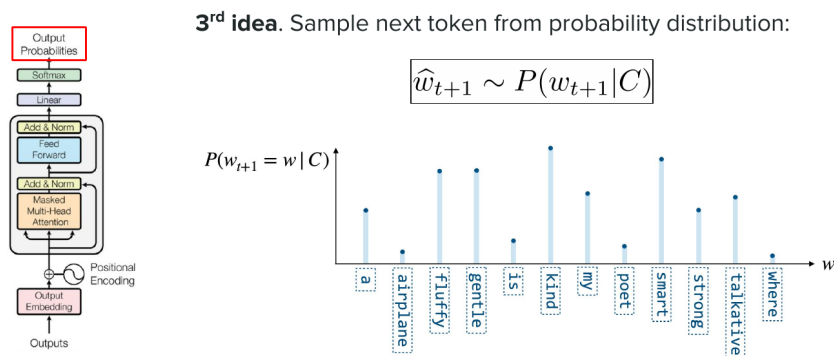
课程也提醒 beam search 并不是今天开放式生成的默认选择，因为它更适合机器翻译等追求高条件概率序列的任务，而开放式对话、创作往往更需要多样性和自然性。

3.3 Sampling、Top-k 与 Top-p

为了保留多样性，生成系统常用 sampling：不是永远取最高概率 token，而是按分布采样。进一步地：

- Top-k：只保留前 k 个最可能 token 后再采样；
- Top-p：保留累计概率达到 p 的最小 token 集合后再采样。

Predicting next token with sampling



Stanford University

"Super Study Guide: Transformers and Large Language Models", Amidi et al., 2024.

ICME

图 7: Sampling 不再固定选择最大概率 token，而是在概率分布上抽样。

这两种方法都在做同一件事：把极低概率、往往会导致胡言乱语的长尾 token 剪掉，同时保留一定随机性。

3.4 Temperature 与 guided decoding

temperature 用来调节分布尖锐程度：

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

符号说明如下：

- z_i ：第 i 个 token 的 logit；
- T ：温度；
- $T < 1$ 时分布更尖锐，更保守；
- $T > 1$ 时分布更平坦，更发散。

Impact of temperature on probabilities

Small T

?

High T

?

Stanford University

ICME

图 8: 温度并不改变 token 排序本身, 而是改变概率分布的尖锐程度。

guided decoding 则是另一条路线: 不是调分布, 而是硬性约束哪些 token 合法。课程用生成 JSON 的例子说明, 如果输出必须满足某种格式, 那么可以在每一步只允许“语法上合法”的 token 进入候选集合。

Constraining the output via guided decoding

Motivation. Generate output in a specific format

Input prompt

*Generate a description of my
33-year old teddy bear who
likes reading.*

Do this in JSON format.

Desired output (JSON)

```
{  
  "first_name": "teddy",  
  "last_name": "bear",  
  "age": 33,  
  "hobby": "reading"  
}
```

Stanford University

ICME

图 9: Guided decoding 通过约束合法 token 集, 强制输出满足预定结构。

3.5 本章小结

模型训练学的是条件分布, 推理系统解决的是“如何从分布中取样”。Greedy、beam、sampling、temperature 和 guided decoding 都在处理这一层。生成质量不仅取决于模型本身, 也取决于解码策略。

4 上下文窗口与提示策略：从 ICL 到 CoT

4.1 Context length 不只是一个数字

课程把 context length 定义为模型在单次前向中能看到的输入长度。它决定了提示词、检索文档、历史对话和中间推理痕迹能否同时放进上下文窗口。随着应用复杂化，context length 直接限制了模型能做多长链条的条件推理。

4.2 In-Context Learning

ICL (In-Context Learning) 指的是不改模型参数，只通过 prompt 中给出的指令和示例，让模型在当前上下文里“表现得像学会了任务”。



Stanford University

Language Models are Few-Shot Learners, Brown et al., 2020.

ICME

图 10: ICL 通过在 prompt 里放入任务说明与示例，让模型在上下文中完成适配。

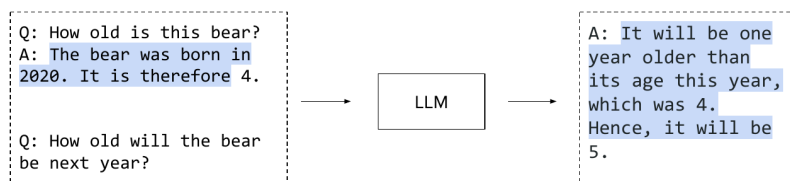
课程把 zero-shot、one-shot、few-shot 都放在这一框架里理解：区别只在 prompt 里是否给例子、给几个例子，而不是模型内部结构发生变化。

4.3 Chain of Thought 与 Self-Consistency

Chain of Thought (CoT) 的核心观察是：对于复杂推理任务，让模型显式生成中间推理步骤，往往比直接输出最终答案更稳。

Chain of thought

Idea. Explaining reasoning helps in improving performance.



Discussion.

- Interpretability + explanation
- More tokens: higher cost and latency

Stanford University

"Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", Wei et al., 2022.

ICME

图 11: CoT 鼓励模型先写出中间推理过程，再给出最终答案。

但 CoT 也会带来更长输出和更高成本，所以课程进一步引出 self-consistency：与其只走一条推理链，不如采样多条 reasoning path，再对最终答案做聚合。这样做的逻辑是，单条链可能偶然走偏，但多个独立样本的共同结论往往更稳。

4.4 本章小结

Prompting 技术的共同点是：不改参数、改输入。ICL 解决快速适配，CoT 解决复杂推理可分解性，self-consistency 则用多路径聚合降低单次采样的偶然性。这些策略解释了为什么“会写 prompt”在 LLM 时代成为真实能力。

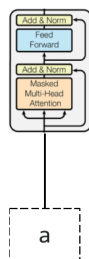
5 推理优化：KV cache、PagedAttention 与更快解码

5.1 为什么推理会慢

课程明确指出，推理阶段的瓶颈往往是 memory-bound，而不是 compute-bound。生成新 token 时，模型需要反复访问之前所有 token 的 key/value；如果每次都从头计算，代价会很高。

Reusing computations with KV caching

Motivation. New token needs to interact with all previous tokens.



Stanford University Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

ICME

图 12: KV caching 通过缓存历史 key/value, 避免每生成一个 token 都重算全部过去。

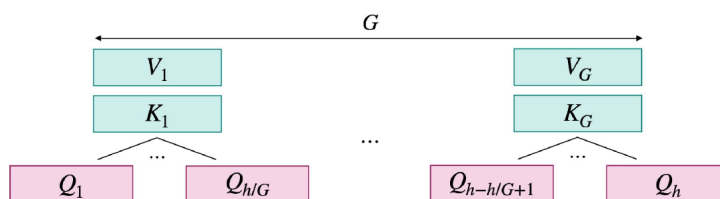
KV cache 的基本思路就是把历史 token 的 key/value 存下来。这样生成第 t 个 token 时, 只需计算最新 token 的投影, 并与缓存交互。

5.2 进一步压缩 KV 的存储成本

缓存历史当然会节省重算, 但也会消耗大量显存。于是课程把上一讲的 MQA/GQA 再拉回来, 说明共享 key/value 头不仅是注意力近似技巧, 也是 KV cache 压缩手段。

Sharing attention heads

Idea. Share key/value attention heads within groups of queries



Stanford University Figure from "Super Study Guide: Transformers & Large Language Models", Amidi et al., 2024.

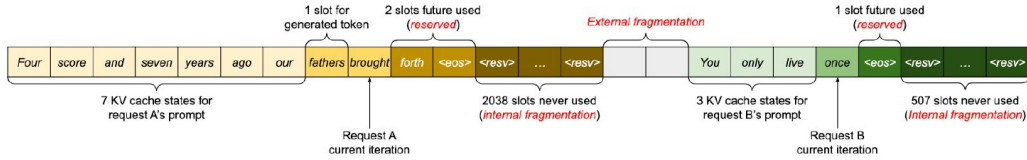
ICME

图 13: MQA/GQA 减少了需要缓存的 key/value 副本数, 从而降低推理内存。

进一步地, PagedAttention 针对服务系统层的内存碎片问题, 把 KV 以分页方式管理, 减少浪费并提高吞吐。

Manage memory with PagedAttention

Observation. Lots of memory waste when storing KV cache.



Stanford University Figure from "Efficient Memory Management for Large Language Model Serving with PagedAttention", by Kwon et al., 2023.

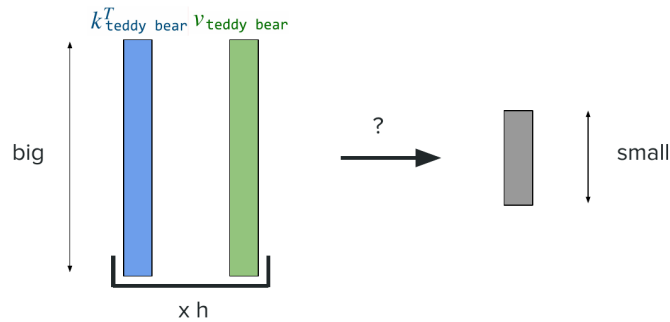
ICME

图 14: PagedAttention 把 KV cache 做成分页管理, 降低服务系统中的显存浪费。

课程还介绍了 latent attention 思路: 不是缓存完整 K/V, 而是缓存更低维的压缩表示, 换取更小内存占用。

Reduce memory of KV cache with latent attention

Solution. Store compressed representations instead!



Stanford University "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model", by DeepSeek, 2023.

ICME

图 15: Latent attention 通过缓存压缩后的表示, 进一步减小 KV cache 的内存占用。

5.3 Speculative decoding 与 Multi-Token Prediction

另一条加速路线不是压 KV, 而是减少大模型逐 token 串行生成的等待时间。Speculative decoding 先让一个小 draft model 连续猜若干 token, 再由大模型一次性验证。如果都被接受, 就相当于一次前向推进多个 token。

Speeding up decoding with speculative decoding

Idea. Use a draft (small) model to generate tokens validated by a target (big) model.

Stanford University "Accelerating Large Language Model Decoding with Speculative Sampling", Chen et al., 2023.

ICME

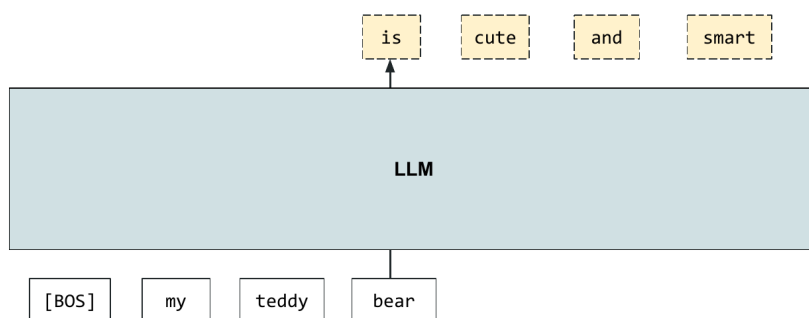
图 16: Speculative decoding 用小模型起草 token，再由大模型并行验证。

课程特别强调它背后的概率论保证：通过接受-拒绝机制，可以让最终采样结果仍然匹配目标大模型分布，而不是仅仅得到一个启发式近似。

Multi-Token Prediction (MTP) 则把“草稿模型”进一步吸收到同一模型内部，通过多个预测头一次预测多个未来 token，再由主头校验。它改变的是训练目标，因此和 speculative decoding 在原理上相近，但不完全等价。

Generate several tokens at once via MTP

MTP = Multi-Token Prediction



Stanford University "Better & Faster Large Language Models via Multi-token Prediction", Gloeckle et al., 2024.

ICME

图 17: MTP 在同一模型中加入多个预测头，以训练时的多 token 目标换取更快生成。

5.4 本章小结

推理优化的主线可以概括为两类：一类是少算或不重算，例如 KV cache、MQA/GQA、latent attention；另一类是更快推进生成步数，例如 speculative decoding 与 MTP。两类方法共同服务于同一个目标：在不明显损伤质量的前提下，提高吞吐、降低延迟。

6 总结与延伸

第三讲把“Transformer 课程”真正推进到了“LLM 课程”。回顾整讲，最值得沉淀的是五条主线：

1. **定义主线**：现代 LLM 通常是大规模 decoder-only 自回归模型。
2. **扩展主线**：MoE 通过稀疏激活扩大总参数规模，而不让单次激活成本同步爆炸。
3. **生成主线**：模型输出的是条件分布，真正的文本风格与稳定性很大程度上由解码策略决定。
4. **提示主线**：ICL、CoT、self-consistency 表明，不改参数也能显著改变任务表现。
5. **系统主线**：当模型足够大时，推理瓶颈进入内存与服务系统层，KV cache、PagedAttention、speculative decoding 等技巧因此成为核心基础设施。

更重要的是，这些内容并不是分散的技巧库。它们一起刻画出今天 LLM 的真实形态：一方面，模型在结构上仍然站在 Transformer 的主干之上；另一方面，真正让它“可用”的，是大量围绕路由、采样、上下文和系统吞吐做出的工程决策。后续继续学习训练、对齐、agent 和评测时，这一讲给出的生成与推理框架会不断反复出现。

4 Lecture 4: LLM training

课程笔记

CME 295 第 4 讲: LLM Training

Codex 基于 Stanford CME 295 第 4 讲视频、字幕、讲稿与 slides 整理

2026 年 4 月 13 日



视频作者/频道: Stanford CME 295

发布日期: 2025 年 10 月 17 日课程录播

视频时长: 1:47:27

视频链接: https://www.youtube.com/watch?v=VlA_jt_3Qc4&list=PLoROMvodv4r0CXd21gf0CF4xr35yINe0y&index=4

目录

1 训练范式：从单任务模型到“预训练 + 调优”	2
1.1 为什么 LLM 训练先学通用能力，再学任务行为	2
1.2 两阶段范式改变了数据、成本和模型复用方式	2
1.3 本章小结	3
2 预训练：让模型先学会语言与代码	3
2.1 预训练的目标是最大化下一个 token 的可预测性	3
2.2 数据混合比单一数据更关键	3
2.3 规模、样本效率与 Chinchilla 法则	3
2.4 预训练为什么又贵又难	4
2.5 本章小结	4
3 训练优化：显存、并行与算子层面的加速	4
3.1 训练瓶颈首先是显存，而不是纸面算力	4
3.2 数据并行和 ZeRO：把冗余状态拆开存	4
3.3 模型并行与训练切分维度	5
3.4 FlashAttention：训练加速不只是分卡，也要减少 HBM 访存	5
3.5 混合精度：用更低精度换更高吞吐和更低显存	6
3.6 本章小结	6
4 监督微调：把“会续写”变成“会配合”	6
4.1 SFT 的角色是让模型学会特定行为	6
4.2 Instruction tuning 的数据比预训练小得多，但质量要求更高	7
4.3 SFT 改变的是行为，不是全部能力边界	7
4.4 SFT 评估仍然很难	7
4.5 本章小结	7
5 参数高效微调：LoRA、QLoRA 与低成本适配	7
5.1 LoRA：不改满矩阵，而是学习一个低秩增量	7
5.2 LoRA 的价值不只是省显存，还在于可插拔	8
5.3 LoRA 应该加在哪里，以及它的训练动态有什么特殊性	8
5.4 QLoRA：把冻结权重量化，再保留全精度 LoRA 分支	9
5.5 QLoRA 说明了“显存预算”本身就是建模约束	10
5.6 本章小结	10
6 总结与延伸	10

1 训练范式：从单任务模型到“预训练 + 调优”

1.1 为什么 LLM 训练先学通用能力，再学任务行为

这一讲先把训练范式讲清楚。传统机器学习里，一个任务往往对应一个模型：垃圾邮件检测训练一个模型，情感分类再训练一个模型，翻译还要单独再来一次。CME 295 强调，语言任务之间并不是完全割裂的，它们共享对语言结构、知识组织、常识与代码模式的理解，所以更合理的做法不是每次从零开始，而是先训练出一个具有广泛语言能力的基础模型，再把它适配到下游任务。

课程把这一过程拆成两段：第一段是 **pretraining**，目标是让模型学会语言与代码的统计结构；第二段是更广义的 **tuning**，目标是让模型在具体使用场景中表现得更像人类需要的系统。这里最重要的思想不是“模型变大了”，而是“训练目标被分层了”：先解决知识与表示能力，再解决任务行为与交互风格。

Paradigm for LLM training

LLM training. Train LLM to understand language, then tune it for end task

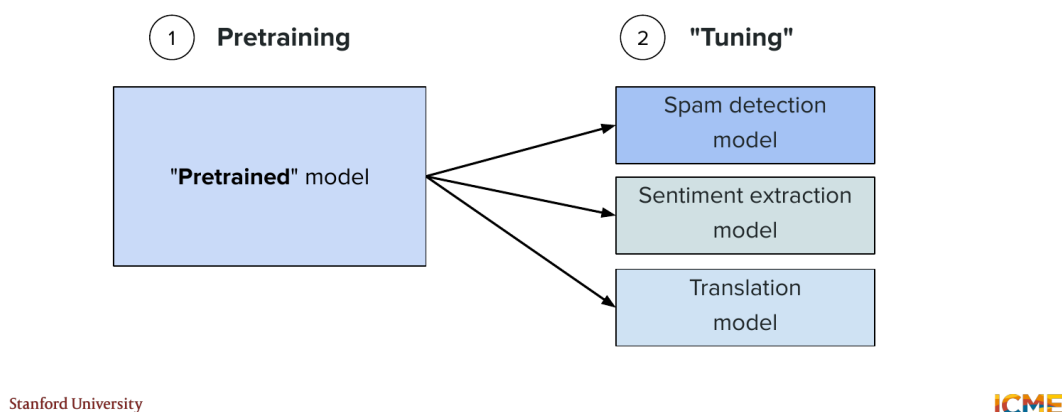


图 1: 课程用两阶段示意图说明 LLM 的主流训练范式：先预训练一个通用模型，再把它适配到具体任务。

1.2 两阶段范式改变了数据、成本和模型复用方式

这个范式带来三层变化。第一，**数据来源改变了**：预训练阶段使用的是规模极大的开放文本、网页和代码混合数据；微调阶段则使用规模更小但质量更高、标签更明确的数据。第二，**成本结构改变了**：真正昂贵的是预训练，它通常消耗大规模 GPU 集群、长时间训练和巨额电力成本；调优虽然仍然昂贵，但一般比预训练小得多。第三，**模型复用方式改变了**：一个预训练模型可以被适配成聊天助手、分类器、翻译器、代码助手，而不必为每个任务重建完整知识底座。

本讲的总纲

Lecture 4 的逻辑是：先解释为什么 LLM 训练要分阶段，再讨论大模型为什么难训练，接着讲硬件和算法层面的训练优化，最后进入 SFT、LoRA 与 QLoRA 这类面向下游适配的方案。

1.3 本章小结

本节的核心不是某个具体公式，而是训练思想的切换。LLM 不再是“一个任务一个模型”，而是“一个基础模型，多种任务适配”。后面所有关于显存、并行、微调和量化的讨论，都是围绕这个分层训练范式展开的。

2 预训练：让模型先学会语言与代码

2.1 预训练的目标是最大化下一个 token 的可预测性

课程把预训练定义得很朴素：给模型大量文本和代码，让它反复做 **next-token prediction**。如果输入序列是 $x_1, x_2, \dots, x_T$ ，那么训练目标可以写成

$$\max_{\theta} \sum_{t=1}^T \log p_{\theta}(x_t | x_{<t}).$$

符号说明如下：

- θ ：模型参数；
- x_t ：序列中第 t 个 token；
- $x_{<t}$ ：第 t 个 token 之前的上下文；
- $p_{\theta}(x_t | x_{<t})$ ：在参数为 θ 的模型下，第 t 个 token 的条件概率。

这里的含义不是让模型“背答案”，而是让它在海量上下文中反复学习语言模式、语法结构、常识关联、代码模板、跨语言映射和领域术语之间的统计关系。课程特别强调，现代 LLM 的预训练语料不只是英文网页，还包含多语言文本和多种编程语言，因此“会写代码”和“会做跨语言推断”并不是额外模块，而是预训练目标在不同分布上的自然外推。

2.2 数据混合比单一数据更关键

Slides 明确给出了预训练数据的几个主要来源：网页抓取数据、百科类知识、代码语料，以及更一般的多语言内容。这里值得注意的是，课程并没有把重点放在“某个单一黄金数据集”，而是放在 **data mixture** 上。原因很直接：如果希望模型既能理解自然语言，又能生成代码，还能在不同语言之间迁移能力，那么训练数据本身就必须覆盖这些分布。

从字幕里的口述可以看出，老师想传达的不是“全部互联网都值得学”，而是“预训练在工程上必须面对大规模、噪声、高异质性数据”。这也是后面训练成本极高、数据治理困难、知识截止与版权问题频繁出现的背景。

2.3 规模、样本效率与 Chinchilla 法则

Slides 用数页内容回顾了 scaling、sample efficiency 和 Chinchilla law。课程的重点不是推导这些论文里的所有细节，而是给学生一个工程直觉：**更大的模型不一定总是更好，关键在于模型参数量与训练 token 量之间是否匹配。**

Kaplan 等人的 scaling laws 告诉我们，模型性能会随着模型规模、数据规模与训练计算量的增加呈现可预测改善；Hoffmann 等人的 Chinchilla 工作进一步指出，在给定训练算力预算下，很多模型其实是“参数太多、token 太少”，因此没有达到计算最优。也就是说，预训练不是盲目堆参数，而是要考虑“模型是否被足够多的数据喂饱”。

2.4 预训练为什么又贵又难

Lecture 4 很早就列出了预训练的几个痛点：

- 成本极高，往往是百万美元量级甚至更高；
- 训练周期长，实验反馈慢；
- 知识天然存在 cutoff，训练结束后世界还在变化；
- 已学到的知识难以局部编辑；
- 数据来源复杂，会触及环境成本与版权争议。

这些挑战说明，预训练并不是“先花钱做完再说”的前置步骤，而是整个 LLM 生命周期中最难逆转、最难局部修补的一步。预训练阶段学到了什么、漏掉了什么、用什么分布学到的，会持续影响后续所有微调与对齐步骤。

关于知识截止的现实含义

预训练模型的知识不是在线更新的数据库，而是训练结束时被压进参数里的静态近似。因此，当世界事实发生变化时，模型可能仍然表现得很自信，但答案已经过时。

2.5 本章小结

预训练的本质是“在海量异质数据上反复做 next-token prediction”，让模型形成通用语言与代码能力。规模法则告诉我们为什么大模型有效，Chinchilla 法则提醒我们不能只堆参数，而成本、知识截止和数据治理问题则解释了为什么预训练是整条链路里最难的一步。

3 训练优化：显存、并行与算子层面的加速

3.1 训练瓶颈首先是显存，而不是纸面算力

进入训练优化部分后，课程先把训练过程中真正占显存的对象列了出来：模型参数、前向激活、反向梯度和优化器状态。对于 Adam 一类优化器来说，参数本身之外还要维护一阶、二阶动量，因此显存压力远大于“只存一份权重”这件事。课程强调，当模型达到数十亿乃至数百亿参数时，单卡显存很快成为首要瓶颈。

3.2 数据并行和 ZeRO：把冗余状态拆开存

最直观的并行方式是数据并行：把 batch 切给多张 GPU，每张卡复制同一份模型，各自做前后向，再聚合梯度。它的优点是逻辑简单，缺点是冗余太多，特别是参数、梯度和优化器状态在每张卡上都来一份。

ZeRO (Zero Redundancy Optimization) 就是为了解决这种冗余。课程按阶段介绍了 ZeRO-1、ZeRO-2、ZeRO-3：

- ZeRO-1 共享优化器状态；
- ZeRO-2 再进一步共享梯度；

- ZeRO-3 连参数本身也做分片。

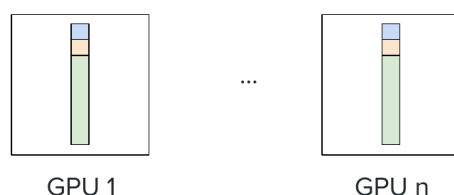
阶段越往后，单卡省下来的显存越多，但通信和实现复杂度也越高。

Data parallelism with ZeRO

Idea. Redundant information across devices

ZeRO = Zero Redundancy Optimization

Variations. ZeRO-3: Share **optimizer state** + **gradients** + **parameters**



Stanford University

"ZeRO: Memory Optimizations Toward Training Trillion Parameter Models", Rajbhandari et al., 2019.

ICME

图 2: ZeRO-3 的核心思想是把优化器状态、梯度和参数都分布到多张 GPU 上，减少数据并行中的冗余副本。

3.3 模型并行与训练切分维度

除了数据并行，课程还列出了模型并行的多种切分方式：tensor parallelism、pipeline parallelism、sequence parallelism、context parallelism 以及专家并行。它们分别从参数维度、层级维度、序列维度或专家路由维度切开模型，让单张 GPU 只承担完整计算图中的一部分。

这里的教学重点是建立“切分维度”的意识：当单卡装不下、单节点带宽不够、序列过长或模型包含稀疏专家结构时，工程师可以选择不同的并行维度，而不是只会加更多显卡做朴素复制。

3.4 FlashAttention：训练加速不只是分卡，也要减少 HBM 访存

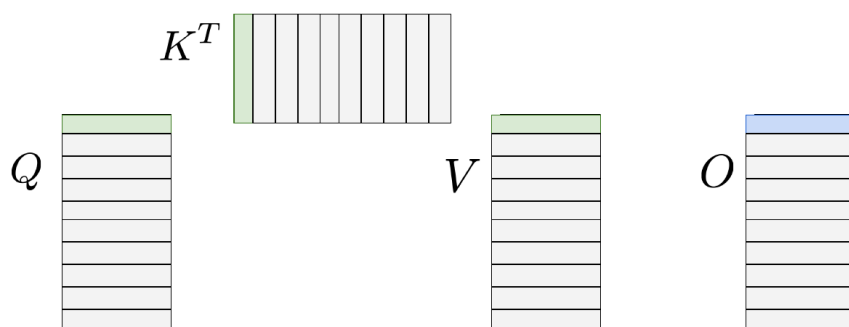
Lecture 4 选了 FlashAttention 作为算子层面优化的代表。课程先回顾了标准 self-attention 的执行路径：把 QK^T 写回 HBM，再从 HBM 读出来做 softmax，再写回概率矩阵，再读出来与 V 相乘。问题在于，HBM 大但慢，反复读写中间矩阵会让 attention 变成明显的 IO-bound 操作。

FlashAttention 的核心思路是：

- 用分块（tiling）策略，把更小的块放进片上 SRAM；
- 在 SRAM 内局部计算并维护 softmax 所需统计量；
- 避免显式物化完整的注意力矩阵与概率矩阵；
- 在反向传播中适度重计算，以换取更低显存占用和更少 IO。

Flash Attention in action

Load blocks of Q , K , V to **SRAM**, compute block of O and write the result to **HBM**



Stanford University "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness", Dao et al., 2022.

ICME

图 3: FlashAttention 的示意图强调了它的关键动作：把 Q 、 K 、 V 分块搬到 SRAM，在片上直接算出输出块，再写回 HBM。

3.5 混合精度：用更低精度换更高吞吐和更低显存

课程后半段还回顾了浮点数表示和 mixed precision training。直觉上，数值精度越低，单个数占的字节越少，硬件每个周期可以处理的元素越多，于是训练速度和显存利用率都会改善。Slides 比较了 FP16、FP32、FP64 和 BF16 的指数位与尾数位差异，帮助学生理解为什么 BF16 在深度学习里尤其常见：它保留了较大的指数范围，同时把位宽压到 16 bit。

课程给出的实践建议是：前向和反向里尽量使用低精度张量以节省资源，但关键权重更新要小心数值稳定性。也就是说，混合精度训练并不是“全部改成低精度”这么简单，而是一个在速度、显存和数值稳定性之间做折中的系统设计。

3.6 本章小结

训练优化的三条主线很清晰：第一，显存压力来自参数、梯度、激活和优化器状态；第二，并行训练通过数据并行、模型并行和 ZeRO 等策略把状态和计算拆散；第三，FlashAttention 和混合精度说明，真正高效的训练还要深入到算子和数值表示层面减少 IO 与内存消耗。

4 监督微调：把“会续写”变成“会配合”

4.1 SFT 的角色是让模型学会特定行为

课程把 SFT (Supervised Fine-Tuning) 定义为：在预训练模型的基础上，用高质量输入输出对继续做 next-token prediction，从而把模型从“会自动补全的语言模型”转化为“愿意按照指令办事的助手”。这一步的重点不在于让模型知道更多世界知识，而在于让模型在给定提示格式下做出人希望看到的回应。

字幕里老师特别强调，预训练后的模型虽然已经懂语言和代码，但仍然更像一个“强大的续写器”，不一定知道如何成为一个有帮助、符合场景预期的系统。SFT 的任务就是教会模型回答问题、

遵循 instruction、给出结构化输出、使用特定语气和任务格式。

4.2 Instruction tuning 的数据比预训练小得多，但质量要求更高

Slides 给出的 instruction tuning 视角很清楚：输入通常是某个任务指令，输出是符合预期的助手回答。对应目标函数仍然是条件语言建模，只不过数据分布变成了“人类希望看到的互动”。课程也明确提到，这类数据既可以由人工撰写，也可以包含合成数据。

与预训练相比，SFT 有三个鲜明特征：

- 数据量小得多，往往是千到百万级样本，而不是万亿 token；
- 数据质量要求更高，因为模型会非常敏感地吸收这些行为模式；
- prompt 分布很重要，数据稍微失衡就可能让模型在某些场景下偏得很厉害。

4.3 SFT 改变的是行为，不是全部能力边界

课程用“能不能把泰迪熊放进洗衣机”这个例子展示了预训练模型和 instruction tuned 模型的区别。预训练模型可能会继续描述材质、结构或中性事实；instruction tuned 模型则更可能直接给出有帮助、可执行、贴合用户意图的建议。这说明 SFT 主要改变的是**行为接口**：同样的知识底座，在不同训练分布下会表现出完全不同的交互风格。

4.4 SFT 评估仍然很难

Slides 在 SFT 部分还列出了常见 benchmark，例如 MMLU、ARC-Challenge、GSM8K 和 HumanEval。它们分别覆盖通识知识、基础推理、数学推理与代码生成等维度。课程用这些 benchmark 传达一个事实：SFT 之后的模型是否“更好”，并不能由单一分数决定，因为模型会在不同维度上呈现不同强弱。

老师还用 Chatbot Arena 一类平台说明“真实用户感受”的重要性。它们能把“哪个模型更顺手、更讨喜”转成某种排序信号，但也存在曝光不均、容易被刷分、用户难以准确判断事实性等问题。也就是说，SFT 评估同时需要离线 benchmark 和在线偏好信号，单靠任一侧都不够。

4.5 本章小结

SFT 的本质不是给模型“再灌一次知识”，而是把预训练模型塑造成某种特定行为接口。它依赖高质量、小规模、分布设计良好的 instruction 数据，能够显著改善交互表现，但评估仍然是一个多维度、难以被单分数概括的问题。

5 参数高效微调：LoRA、QLoRA 与低成本适配

5.1 LoRA：不改满矩阵，而是学习一个低秩增量

Lecture 4 最后把重点放在参数高效微调（PEFT）上，其中 LoRA 是主角。它的核心思想是：冻结原始权重 W_0 ，只学习一个低秩增量 BA ，于是新的线性层写成

$$W = W_0 + BA.$$

符号说明如下：

- W : 微调后的有效权重;
- W_0 : 冻结的原始预训练权重;
- B, A : LoRA 额外学习的低秩矩阵;
- BA : 对原始权重的低秩增量修正。

其中 $B \in \mathbb{R}^{d \times r}$ 、 $A \in \mathbb{R}^{r \times k}$ ，而秩 r 远小于原矩阵维度。这样一来，真正要训练的参数数量就从完整矩阵降到了两个小矩阵。

这背后的工程直觉很重要：很多下游任务不需要完全重写全部表示，只需要在某些方向上对原有权重做可控偏移。LoRA 假设这种偏移可以被低秩结构很好近似，于是把训练成本降了下来。

5.2 LoRA 的价值不只是省显存，还在于可插拔

Slides 专门展示了“swap matrices = swap tasks”的画法，意思是：底层基础模型不变，只需替换不同任务对应的 LoRA adapter，就能让同一个基座承担不同功能。这种可插拔能力非常适合多任务部署，也让分享和分发模型变得更轻量。

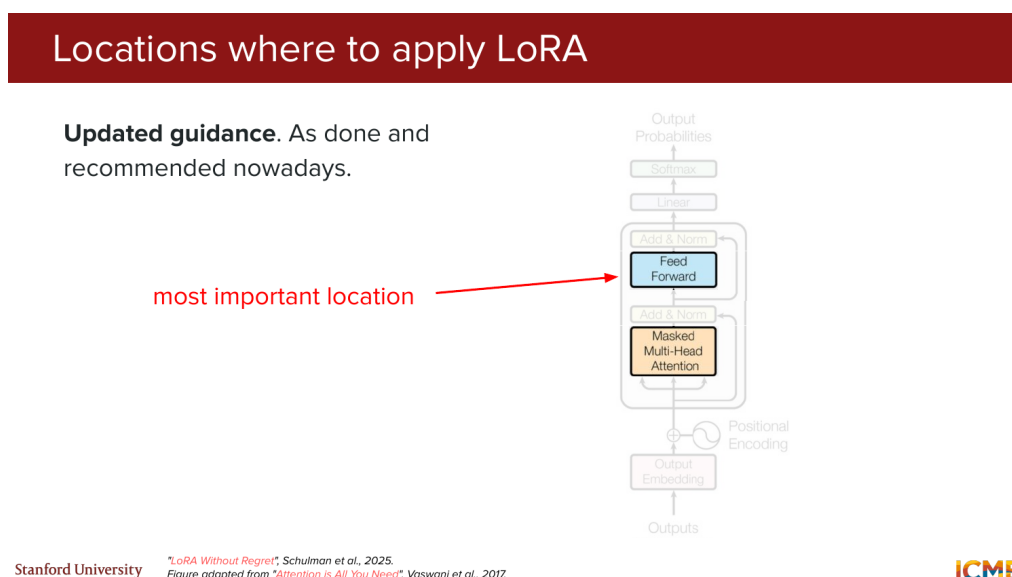


图 4: 课程给出的更新版建议强调：LoRA 通常最值得优先放在前馈层等高影响位置，而不是无差别地加到所有线性层。

5.3 LoRA 应该加在哪里，以及它的训练动态有什么特殊性

课程对 LoRA 的讲解没有停留在定义层面，而是进一步讨论“放在哪里更有效”。Slides 对比了原始论文中的做法与更新后的经验法则，指出如今更常见也更推荐的选择，是优先覆盖最关键、对表示能力影响最大的层，尤其是前馈网络中的某些权重。

老师还点出两条容易被忽略的训练动态：

- LoRA 往往需要比 full fine-tuning 更高的学习率；
- 在很大的 batch 下，LoRA 的表现可能不如 full fine-tuning 稳定。

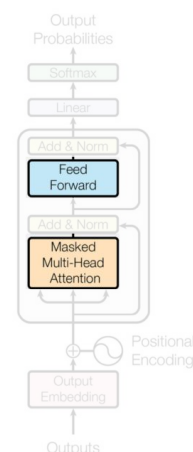
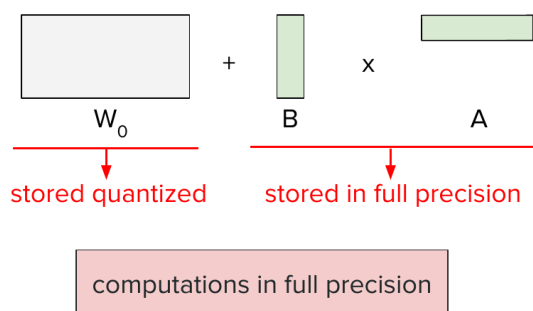
这提醒我们，LoRA 不是“无脑替代品”，而是一个带有自身超参数规律和优化习惯的训练方法。

5.4 QLoRA: 把冻结权重量化，再保留全精度 LoRA 分支

当显存继续成为主要瓶颈时，课程引入了 QLoRA。它延续 LoRA 的结构，但进一步把冻结的底座权重做量化存储，只把 LoRA 的小矩阵保留在全精度里参与训练，于是同时获得两种好处：底座权重占用更少显存，adapter 训练仍然保持较好数值表现。

QLoRA

Idea. Quantize all frozen weights to relieve memory bottleneck.



Stanford University

QLoRA: Efficient Finetuning of Quantized LLMs, Dettmers et al., 2023.
Figure adapted from "Attention is All You Need", Vaswani et al., 2017.

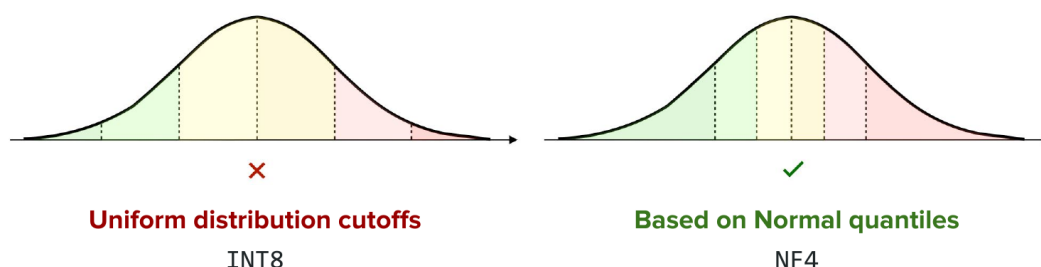
ICME

图 5: QLoRA 的结构示意：底座权重以量化形式存储，LoRA 分支保留全精度，前向计算仍在高精度路径中完成。

课程还解释了两个关键技巧。第一个是 **NF4**，即 4-bit NormalFloat。它不是简单做均匀切分，而是依据近似正态分布的分位点来设置量化区间，更贴近神经网络权重的统计形状。第二个是 **double quantization**，即连量化常数本身也继续压缩，进一步减少额外开销。

Efficient quantization

Trick. Use 4-bit NormalFloat (NF4) to best split the space



Stanford University

Figure from "Super Study Guide: Transformers and Large Language Models", Amidi et al., 2024.

ICME

图 6: NF4 的直觉是按近似正态分布的分位点切分数值范围，而不是均匀切分，从而更有效利用 4 bit 表示能力。

5.5 QLoRA 说明了“显存预算”本身就是建模约束

Slides 给出了数量级上的收益：在 LLaMA 65B 一类模型上，QLoRA 能把微调显存需求降到一个更可操作的水平，同时 double quantization 还能再带来额外节省。课程希望学生记住的不是某个精确百分比，而是更一般的原则：一旦显存预算成为硬约束，模型适配方法本身就必须围绕显存来设计。

LoRA 与 QLoRA 的关系

LoRA 解决的是“只训练少量增量参数”，QLoRA 解决的是“连冻结底座也要尽量省显存”。前者强调参数高效，后者强调参数高效加存储高效。

5.6 本章小结

LoRA 把完整微调改写成低秩增量学习，适合在共享基座上快速适配多个任务；QLoRA 则进一步量化冻结权重，让大模型在更小硬件预算上也能完成高质量微调。两者共同说明，下游适配已经不只是算法问题，更是资源约束下的系统设计问题。

6 总结与延伸

Lecture 4 串起了 LLM 生命周期中的第一批关键环节。首先，训练范式从“单任务单模型”转向“预训练 + 调优”的两阶段结构，解释了为什么一个基础模型可以服务多种任务。其次，预训练之所以昂贵，不只是因为模型大，更因为它要在庞大异质数据上长期优化 next-token objective，同时面对规模律、数据混合、知识截止与成本治理等现实问题。再次，真正把大模型训起来，必须处理显存瓶颈、分布式并行、FlashAttention 与混合精度等系统层问题。最后，SFT、LoRA 与 QLoRA 则把注意力从“如何学到通用能力”转向“如何以更低成本把能力塑造成产品可用的行为”。

从课程安排上看，这一讲实际上是在为后两讲铺路。SFT 会自然过渡到 preference tuning，LoRA 和 QLoRA 也为后来讲到的 RLHF、DPO、reasoning model training 提供了现实可行的训练工具。如果把 Lecture 4 压缩成一句话，那就是：**现代 LLM 训练既是统计学习问题，也是显存、带宽、并行和部署预算共同约束下的系统工程。**

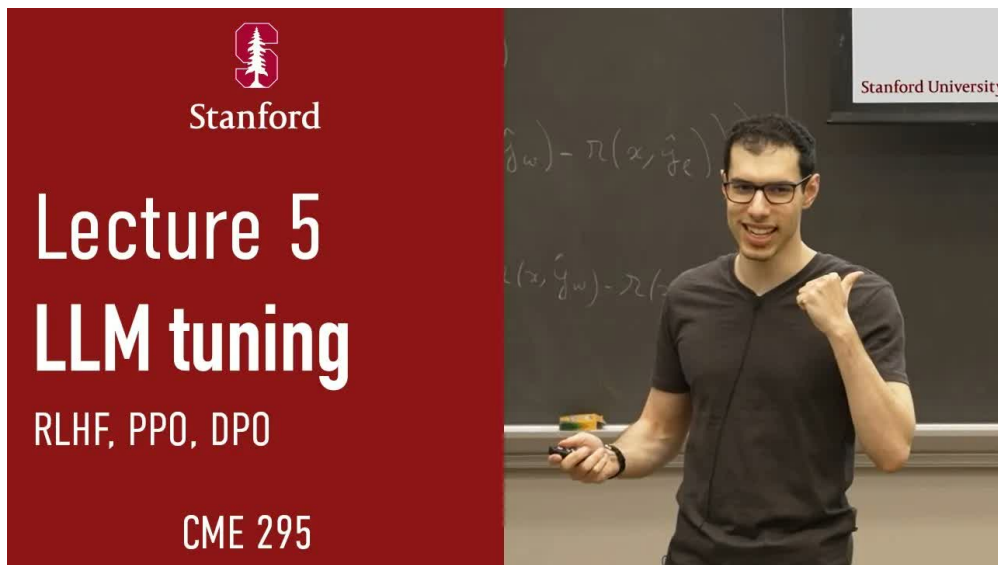
5 Lecture 5: LLM tuning

课程笔记

CME 295 第 5 讲: LLM Tuning

Codex 基于 Stanford CME 295 第 5 讲视频、字幕、讲稿与 slides 整理

2026 年 4 月 13 日



视频作者/频道: Stanford CME 295

发布日期: 2025 年 10 月 31 日课程录播

视频时长: 1:47:41

视 频 链 接: https://www.youtube.com/watch?v=PmW_TMq3l0I&list=PLoROMvodv4rOCXd21gf0CF4xr35yINe0y&index=5

目录

1 为什么 SFT 之后还需要 Preference Tuning	2
1.1 SFT 能教模型行为，但不一定能稳定注入“负信号”	2
1.2 为什么偏好数据往往比 SFT 数据更容易收集	2
1.3 本章小结	2
2 偏好数据：如何把“更好”变成可训练的监督	2
2.1 Pointwise、pairwise、listwise 三种视角	2
2.2 课程给出的 pairwise 数据采集 recipe	3
2.3 Bradley-Terry 视角：把“更好”写成概率	3
2.4 本章小结	4
3 RLHF：先学奖励模型，再用强化学习对齐策略	4
3.1 RLHF 的两阶段结构	4
3.2 奖励模型学到的是“好坏判别器”	4
3.3 把 LLM 写进 RL 框架	5
3.4 本章小结	5
4 PPO、挑战与 Best-of-N：当 RL 真正落地时会遇到什么	5
4.1 PPO 的直觉：既追奖励，又别偏离基座太远	5
4.2 优势函数为什么重要	6
4.3 PPO 为什么在工程上很重	6
4.4 Best-of-N：如果不做 RL，还能怎样利用奖励模型	6
4.5 本章小结	7
5 DPO：把偏好学习重新写回监督学习	7
5.1 DPO 的出发点	7
5.2 DPO 损失在优化什么	8
5.3 DPO 与 RLHF 的取舍	8
5.4 本章小结	9
6 总结与延伸	9

1 为什么 SFT 之后还需要 Preference Tuning

1.1 SFT 能教模型行为，但不一定能稳定注入“负信号”

Lecture 5 一开始先回顾前一讲：预训练让模型具备语言和代码能力，SFT 让模型学会某种期望行为，但这还不够。课程给出的关键动机是，模型即使已经经过 SFT，仍然可能在语气、安全性、帮助性或偏好上“行为不对劲”。例如同样是建议和泰迪熊做什么活动，模型可能给出并不友好、也不符合人类期待的回答。

老师强调，这类问题并不总能通过继续往 SFT 数据里追加“正确答案”来解决。因为 SFT 更像是明确教模型“应该生成什么”，而 preference tuning 更适合告诉模型“这类输出比另一类输出更好”。也就是说，SFT 擅长正向示范，preference tuning 擅长引入排序和负信号。

1.2 为什么偏好数据往往比 SFT 数据更容易收集

课程给出一个很实用的人类认知视角：让标注者从零写出高质量答案很难，但让标注者在两个答案之间选一个更好，通常容易得多。对于真实系统也是如此，很多时候人类产品团队并不容易直接写出完美回答，但很容易判断“回答 A 比回答 B 更安全、更有帮助、更像我们想要的风格”。

此外，字幕里的讲解还特别提醒：SFT 数据分布非常敏感，一旦不断往数据里补某一类 prompt，模型就可能被过度拉向某个局部行为分布。preference tuning 则提供了一种更柔性的方式，通过“偏好比较”而不是“硬编码唯一正确输出”来修正模型行为。

Lecture 5 的出发点

这一讲不是在否定 SFT，而是在补足 SFT。课程想说明的是：要让模型不仅会回答，还要会按照人类偏好回答，排序信号和奖励信号就变得必要。

1.3 本章小结

Lecture 5 的问题意识非常明确：SFT 负责把模型变成可用助手，preference tuning 负责把助手变成更符合人类偏好的助手。两者不是替代关系，而是能力塑形链条上的前后两步。

2 偏好数据：如何把“更好”变成可训练的监督

2.1 Pointwise、pairwise、listwise 三种视角

Slides 把 preference data 统一写成观测 $(x, \hat{y})$ ，也就是 prompt 和模型回答。接着课程介绍了三种标注思路：

- pointwise：给单个回答打分；
- pairwise：比较两个回答谁更好；
- listwise：给一组回答排序。

老师的判断很明确：**pairwise 是最实用的主流方案**。pointwise 最大的问题是分数尺度不稳定，很难保证不同标注者对‘0.9’和‘0.4’的理解一致；listwise 信息更丰富，但认知负担更高。pairwise 则在信息量和可操作性之间取得了很好的平衡。

Preference data

Observation = (prompt x , response $\hat{y}$)

Pointwise		Pairwise		Listwise	
Obs 1	0.4	Obs 1 < Obs 2		Obs 2	1
Obs 2	0.9	Obs 1 > Obs 3		Obs 1	2
Obs 3	0.1	Obs 1 > Obs 4		Obs 4	3
Obs 4	0.2	Obs 2 > Obs 3		Obs 3	4

Stanford University

ICME

图 1: 课程把偏好数据分成 pointwise、pairwise 与 listwise 三类, 并强调 pairwise 是现实中最常用的折中方案。

2.2 课程给出的 pairwise 数据采集 recipe

Lecture 5 没有停留在概念层面, 而是给出一套很具体的 recipe。第一步, 对同一个 prompt 生成多个回答。输入 prompt 既可以来自用户日志, 也可以来自某个参考分布; 候选回答可以来自 SFT 模型采样、合成重写, 或者更一般的候选生成机制。第二步, 对这些候选进行标注, 得到“赢者/输者”。

这个流程里最关键的点在于: **偏好数据不是凭空来的, 而是从候选生成和人类判断共同构成的流水线里产出的**。换句话说, 偏好训练的上限很大程度上取决于候选回答是否足够多样、标注标准是否稳定, 以及 prompt 分布是否贴近真实应用。

2.3 Bradley-Terry 视角: 把“更好”写成概率

为了把 pairwise preference data 变成可优化目标, 课程引入了 Bradley-Terry 形式。若给定 prompt x , 回答 y_w 比回答 y_l 更受偏好, 那么可以写成

$$P(y_w \succ y_l | x) = \frac{\exp(r(x, y_w))}{\exp(r(x, y_w)) + \exp(r(x, y_l))}.$$

这个公式表达的是: 比较的不是绝对真值, 而是两个候选回答的相对奖励大小。

符号说明如下:

- x : 输入 prompt;
- y_w : 被偏好的回答, 也就是 “winner”;
- y_l : 不被偏好的回答, 也就是 “loser”;
- $r(x, y)$: 模型或奖励函数给回答分配的标量分数;
- $P(y_w \succ y_l | x)$: 在给定 prompt 下, 赢者比输者更优的概率。

这一步的意义在于，偏好数据终于从“人工主观比较”转成了“可学习的概率建模对象”，从而为 reward model 或 DPO 这类方法打下了统一基础。

2.4 本章小结

偏好数据的核心不是给出唯一标准答案，而是把人类判断转成相对排序。pairwise 标注因此成为最常用方案，而 Bradley-Terry 形式则进一步把“谁更好”变成了一个能被优化的概率目标。

3 RLHF：先学奖励模型，再用强化学习对齐策略

3.1 RLHF 的两阶段结构

RLHF 的全称是 Reinforcement Learning from Human Feedback。课程把它拆成两个阶段：第一阶段训练奖励模型，输入是 prompt 和回答，输出是一个可比较的分数；第二阶段用这个奖励信号去更新当前策略模型，让它更偏向高分回答。

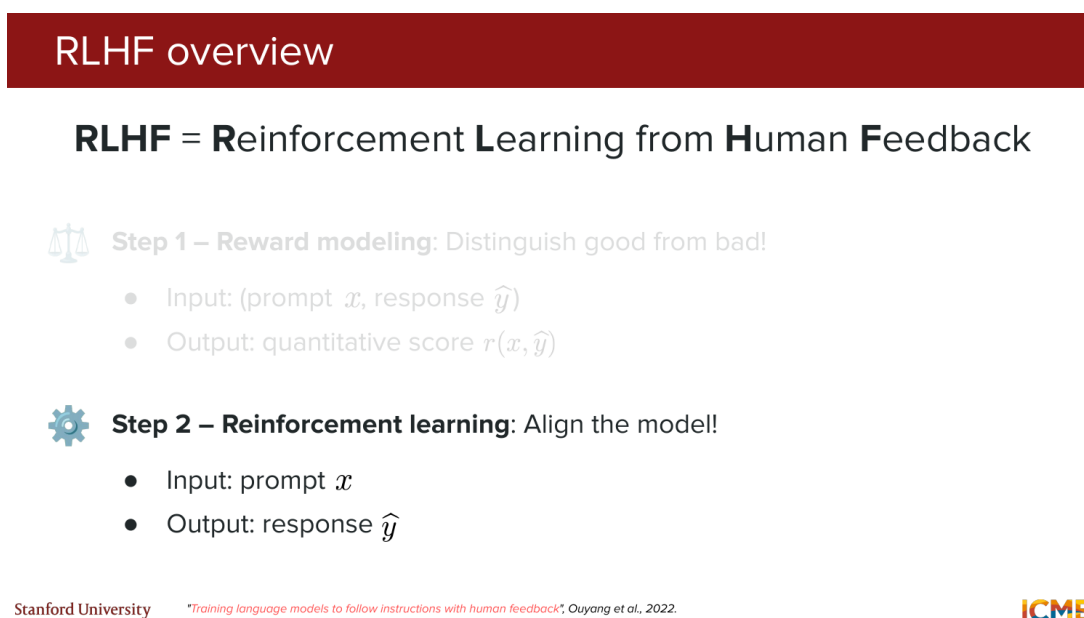


图 2: RLHF 的基本结构是两步：先训练 reward model，再用强化学习把策略朝更优回答推进。

这两阶段结构非常重要，因为它说明 RLHF 不是“直接拿人工偏好去训语言模型”，而是先把人工偏好压缩成奖励函数，再把语言模型放进 RL 框架里迭代优化。

3.2 奖励模型学到的是“好坏判别器”

课程在 reward modeling 里强调，奖励模型本质上是一个判别器。它不负责生成回答，而负责对候选回答打分，告诉后续 RL 哪种输出更值得鼓励。Slides 指出，这一步通常使用数量级为 $O(10^4)$ 的人工偏好样本，并且经常是在一个预训练语言模型上加分类头来完成。

这一阶段的目标不是彻底解决生成问题，而是构造一个足够稳定的“偏好代理”。如果这个代理本身学偏了，那么第二阶段的 RL 就会把偏差放大。于是，reward model 的质量直接决定 RLHF 后半段的训练方向。

3.3 把 LLM 写进 RL 框架

Lecture 5 还把经典 RL 设置重新映射到 LLM 上：agent 就是语言模型，state 是当前为止的输入与已生成 token，action 是下一个 token，policy 是下一个 token 的概率分布，reward 则来自人类偏好或奖励模型。这个映射有助于学生理解，为什么语言模型也可以被看成一个逐 token 决策系统。

在这个视角下，RLHF 并不是魔法，而是把语言建模问题改写成序列决策问题。模型每多生成一个 token，就相当于在当前状态下做了一次动作选择；整个回答结束后，再根据完整输出获得奖励或优势信号。

3.4 本章小结

RLHF 的结构可以浓缩成一句话：先学一个会打分的偏好代理，再让语言模型对着这个代理优化。它的强大之处在于能利用排序信号持续调整策略，但也因此天然是一个多阶段、较重的系统。

4 PPO、挑战与 Best-of-N：当 RL 真正落地时会遇到什么

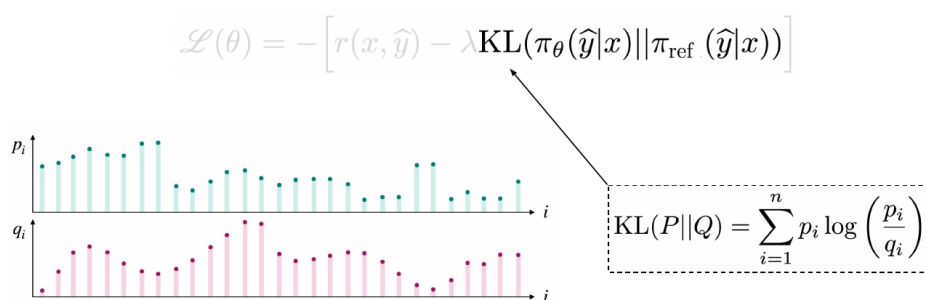
4.1 PPO 的直觉：既追奖励，又别偏离基座太远

课程把 PPO 作为 RLHF 中最经典的策略优化算法来讲。Slides 的核心信息是：优化目标不能只追求更高 reward，否则模型会迅速偏离原本已经学到的有用行为模式，出现 reward hacking 或训练不稳定。因此 PPO 需要同时处理两件事：

- 让高优势回答更容易被生成；
- 约束新策略不要离旧策略或参考策略太远。

Common RL algorithm: PPO

PPO = Proximal Policy Optimization



Stanford University Figure from "Super Study Guide: Transformers and Large Language Models", Amidi et al., 2024.

ICME

图 3: PPO 在课程中的呈现重点是“奖励提升”和“KL 约束”并存：模型要变好，但不能失控。

课程还特别提醒学生，PPO 里常见的“奖励”并不总是原始 reward，本质上优化的是 advantage，也就是“比基线好多少”。

4.2 优势函数为什么重要

课程在后续 slides 中解释了 advantage 的角色。若直接使用 reward，梯度方差会很大，训练不稳定；因此通常会减去一个 baseline，得到更稳定的优势估计：

$$A = R - b.$$

这里的含义是，如果某次输出虽然奖励高，但本来就很容易得到，那它不值得被过度放大；真正值得强化的是“比预期更好”的那部分。

符号说明如下：

- A : advantage，表示当前输出比基线好多少；
- R : 最终奖励，可以来自 reward model；
- b : baseline，常由 value function 或其他估计器提供。

这种设计直接服务于训练稳定性，也是 PPO 区别于“只追求高分”的简单方法的关键。

4.3 PPO 为什么在工程上很重

Lecture 5 对 PPO 的批评非常具体。Slides 列出了几个现实缺点：通常需要政策模型、value 模型、reward 模型和参考模型四套角色；超参数很多；训练过程容易不稳定；监控指标也不好选。此外，还要保证采样足够多样，否则模型容易被窄分布输出困住。

这些问题解释了为什么 RLHF 很强，但代价也很高。它不仅是“再训练一次模型”，而是搭起了一整套相互依赖的优化系统。

4.4 Best-of-N：如果不做 RL，还能怎样利用奖励模型

课程接着给出一个很务实的 workaround：Best-of-N (BoN)。思想很简单，不更新策略模型，而是在推理时对同一 prompt 生成 N 个候选回答，用 reward model 给它们打分，再选出最好的那个。于是流程变成：

- 给定 prompt；
- 用 SFT 模型生成多个候选；
- 用 reward model 排序；
- 取最高分回答。

Workaround if we don't want to do RL

BoN = Best of N

Idea. Skip the RL step and leverage the reward model scores

Strategy.

- Given a prompt, generate several outputs with SFT model
- Rank output with score given by reward model
- Take the best one

Stanford University

ICME

图 4: Best-of-N 的核心是跳过 RL 训练，把 reward model 用在推理阶段的候选重排上。

BoN 之所以重要，不是因为它最终一定最好，而是因为它揭示了“训练时改策略”和“推理时改选择”之间的替代关系。课程后面再引出 DPO，就是在继续探索能否把偏好学习重新拉回监督学习范式。

4.5 本章小结

PPO 说明 RLHF 的核心难点不只是目标定义，还包括训练稳定性、模型数量和系统复杂度。BoN 则提供了一个不做 RL 的替代方向，为后面的 DPO 铺路。

5 DPO：把偏好学习重新写回监督学习

5.1 DPO 的出发点

Lecture 5 后半部分的核心转折是 Direct Preference Optimization。它的问题非常直接：既然 RLHF 太重、BoN 又把成本挪到了推理期，那能不能直接在偏好数据上做一种监督式优化，而不显式训练 reward model 和 value model？DPO 的答案是可以。

Supervised approach with DPO

DPO = Direct Preference Optimization

Rewrite the **loss function** in a supervised way:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- No need to train a separate reward model $r_{\theta}(x, y) = \beta \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$
- Operates directly on preference data
- Similar to the Bradley-Terry formulation with a special kind of reward!

Stanford University

"Direct Preference Optimization: Your Language Model is Secretly a Reward Model", Rafailov et al., 2023.

ICME

图 5: DPO 把偏好学习重新表达成监督目标：直接比较赢者和输者在当前策略与参考策略下的相对概率。

5.2 DPO 损失在优化什么

课程把 DPO 解释成一种“特殊形式的 Bradley-Terry + 参考模型对比”。其典型目标可写成

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right).$$

这意味着，模型被鼓励在参考模型的基准上，提高对赢者回答的相对偏好，同时压低对输者回答的相对偏好。

符号说明如下：

- $\mathcal{D}$: 偏好数据集；
- x : prompt；
- y_w, y_l : 偏好比较中的赢者与输者回答；
- π_{θ} : 当前待训练策略；
- π_{ref} : 参考策略，通常是基座或 SFT 模型；
- β : 控制偏好强度的超参数；
- $\sigma(\cdot)$: sigmoid 函数。

从教学角度看，课程最想让学生理解的是：DPO 并没有否定 RLHF 的思路，而是把 RLHF 中隐含的奖励结构重新改写成一个更接近监督学习的目标。

5.3 DPO 与 RLHF 的取舍

Slides 最后专门做了对比。DPO 的好处是流程更简单，不需要单独维护 reward model 和 value model；代价是它仍然依赖偏好数据质量，并且不同任务上是否优于 PPO 没有统一结论。课程因此

并没有把 DPO 讲成“全面替代 RLHF 的答案”，而是把它定位为一类更轻、更容易实现、但仍需具体问题具体分析的方案。

课程给出的结论很克制

在实现复杂度上，DPO 明显比 PPO-based RLHF 轻；但在最终效果上，没有普适的绝对赢家，很多结论都依赖任务、数据与工程实现细节。

5.4 本章小结

DPO 的价值在于，它把偏好学习重新拉回监督学习框架，让训练流程更轻、更直接；但这不等于 RLHF 就过时了。Lecture 5 真正建立的是一张方法地图：从 pairwise data 到 reward model，再到 PPO、BoN 和 DPO，每一种方法都在不同位置权衡了训练成本、推理成本和实现复杂度。

6 总结与延伸

Lecture 5 完成了从“模型会遵循指令”到“模型更符合人类偏好”的过渡。课程首先说明，SFT 不能自然提供足够稳定的负信号，因此需要偏好比较；接着用 pairwise 数据和 Bradley-Terry 形式把“更好”变成可训练对象；然后引出 RLHF 的两阶段结构，并以 PPO 为代表讲清楚为什么真实对齐训练必须在奖励提升与策略约束之间做平衡。随后，BoN 展示了不用 RL 也能利用 reward model 的推理期方案，而 DPO 则把偏好学习重新表达成监督优化，显著降低了系统复杂度。

如果把 Lecture 4 和 Lecture 5 连起来看，可以得到一条很清楚的能力塑形路径：预训练学通用知识，SFT 学任务行为，preference tuning 学人类偏好。再往下一讲，课程就会进一步把这些偏好优化和 RL 技术迁移到 reasoning model 的训练上，讨论如何让模型不仅“更像人”，还“更会想”。

6 Lecture 6: LLM reasoning

课程笔记

CME 295 第 6 讲: LLM Reasoning

Codex 基于 Stanford CME 295 第 6 讲视频、字幕、讲稿与 slides 整理

2026 年 4 月 13 日



视频作者/频道: Stanford CME 295

发布日期: 2025 年 11 月 7 日课程录播

视频时长: 1:47:07

视 频 链 接: <https://www.youtube.com/watch?v=k5Fh-UgTuCo&list=PLoROMvodv4rOCXd21gf0CF4xr35yINe0y&index=6>

目录

1 从 Vanilla LLM 到 Reasoning Model：问题到底变了什么	2
1.1 为什么课程先批判“普通 LLM”的边界	2
1.2 课程对 reasoning 的工作定义	2
1.3 新范式：输出不再只有答案，还包含推理链	2
1.4 本章小结	3
2 评测与表征：怎样判断模型真的更会推理了	3
2.1 Reasoning 任务最常见的两大载体：代码和数学	3
2.2 Pass@k 与 Cons@k：为什么 reasoning 评测经常不是单次命中率	3
2.3 课程为什么强调 reasoning model 是近一年才爆发的主题	3
2.4 本章小结	3
3 可验证奖励与 Test-Time Scaling：让模型愿意多想、还能被检查	4
3.1 为什么 reasoning chain 不能完全依赖人工写 SFT 数据	4
3.2 可验证奖励的核心：格式 + 正确性	4
3.3 Test-time scaling 的真正含义	5
3.4 本章小结	5
4 GRPO 与长度偏置：Reasoning RL 为什么和 PPO 不一样	5
4.1 GRPO 的核心改动：用组内平均回报做基线	5
4.2 为什么 GRPO 很适合 reasoning 场景	6
4.3 输出长度为什么会越训越长	6
4.4 DAPO、Dr. GRPO 与其他修正	7
4.5 本章小结	7
5 DeepSeek R1 系列与 Distillation：把推理能力真正做成一条训练流水线	7
5.1 R1-Zero：只靠 RL 就能逼出推理行为，但还不够产品化	7
5.2 R1：把 SFT 与 RL 重新交错起来	8
5.3 Distillation：把 reasoning traces 蒸馏给更小模型	8
5.4 本章小结	9
6 总结与延伸	9

1 从 Vanilla LLM 到 Reasoning Model: 问题到底变了什么

1.1 为什么课程先批判“普通 LLM”的边界

Lecture 6 并不是一上来就讲 DeepSeek R1，而是先花不少时间说明 vanilla LLM 的强项和短板。课程承认，普通 LLM 已经非常擅长模仿、续写、生成代码和修 bug，也能在很多开放任务上给出看似不错的回答；但它们仍然存在几类明显问题：推理能力有限、知识静态、无法直接执行动作，而且自由文本输出很难评价。

老师这样铺垫的目的，是把“reasoning model”从营销标签还原成一个技术问题：如果普通 LLM 的训练目标主要是在已有分布上生成高似然文本，那么为什么它会天然擅长多步求解复杂问题？答案当然是不会。因此 reasoning model 不是普通助手的简单加强版，而是围绕“多步求解”重新组织训练与评测目标的产物。

1.2 课程对 reasoning 的工作定义

Slides 明确说，并没有一个全行业统一认可的 reasoning 定义。课程给出的工作定义是：**reasoning 是解决问题的能力，尤其是需要多步中间过程的问题**。这里典型例子是数学题、代码题、逻辑题，而不是直接从记忆里取出一个事实。

老师还给出了一组对照：像“Stanford 这门课的课程代码是什么”更像是检索式问题，而“这只熊 2020 年出生，现在 2025 年，几岁了”则需要显式中间步骤。这个区分非常关键，因为它说明 reasoning 并不等同于“回答正确”，而是和问题是否需要分解、推演、验证直接相关。

1.3 新范式：输出不再只有答案，还包含推理链

课程把 reasoning model 的核心转向画得很直白：过去是 Question -> LLM -> Answer，现在变成 Question -> LLM -> Reasoning chain + Answer。也就是说，模型被鼓励先产生中间推理，再给最终答案。

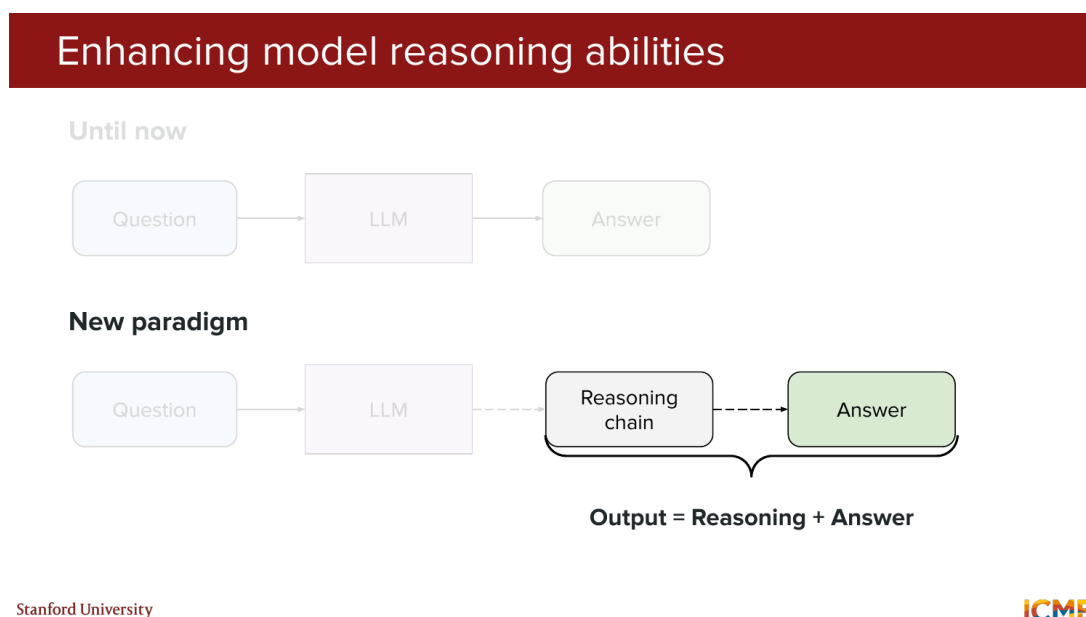


图 1: Reasoning model 的新范式：输出不仅包含最终答案，还显式包含中间 reasoning chain。

这一变化的技术意义非常大。它把原本隐含在模型内部的“求解过程”变成显式可训练、可奖励、可控制的对象。后面所有 test-time scaling、verifiable reward、GRPO 和 R1 训练 recipe，都是在围绕这条“让推理链可优化”展开。

1.4 本章小结

Lecture 6 的第一步是澄清问题：reasoning model 不是“更强聊天机器人”这么简单，而是把多步求解过程显式化的模型。只有先承认 vanilla LLM 的能力边界，后面关于 reasoning training 的设计才有意义。

2 评测与表征：怎样判断模型真的更会推理了

2.1 Reasoning 任务最常见的两大载体：代码和数学

Slides 给出的 benchmark 景观非常典型。代码方向上，课程提到 HumanEval、CodeForces、SWE-bench；数学方向上则提到 AIME、GSM8K 等。这些任务共同特点是：它们都要求模型不是简单“说得像”，而是要真的给出可检查的结果，比如通过测试用例、得到正确数值答案、或完成逻辑推导。

从教学角度看，课程想让学生注意的是：reasoning benchmark 往往有某种**可验证性**。这和开放式聊天质量不同，因为系统可以较客观地判断答案是否正确，也因此更适合作为 RL 奖励的来源。

2.2 Pass@k 与 Cons@k：为什么 reasoning 评测经常不是单次命中率

Lecture 6 特别回顾了 code benchmark 里常见的 Pass@k。其含义是：如果允许模型尝试 k 次，至少有一次成功的概率是多少。对于能自动验证的任务，这个指标非常合理，因为真实系统可以多采样、跑测试、再选最好结果。

Slides 还提到 Pass@1 与 Cons@k。Pass@1 对应“只生成一次就要尽量答对”的场景；Cons@k 则强调多次采样后的一致性 or 共识。这些指标共同说明，reasoning model 的评测和普通聊天模型不同，它不一定只关心单次输出，而经常关心“多次尝试 + 可验证选择”这一整套流程。

Lecture 6 的评测观

如果一个任务的验证器足够可靠，那么系统不必把全部赌注押在一次采样上。Pass@k 之类指标正是把“可多次尝试、可自动检查”的现实 workflow 编码进评测方式里。

2.3 课程为什么强调 reasoning model 是近一年才爆发的主题

字幕里老师反复提到，这一讲的大部分材料都来自 2024 或 2025 年，非常新。这一提醒有两层含义：第一，很多方法仍在快速演化，没有最终定论；第二，现在回头看已经能分辨哪些东西只是热闹，哪些东西真的构成了 reasoning training 的稳定骨架。Lecture 6 选择聚焦的，就是后者。

2.4 本章小结

Reasoning model 的评测通常依赖代码、数学这类可验证任务，指标也往往从单次准确率扩展到 Pass@k、Cons@k 等多采样视角。这为后续引入“可验证奖励”提供了非常自然的桥梁。

3 可验证奖励与 Test-Time Scaling: 让模型愿意多想、还能被检查

3.1 为什么 reasoning chain 不能完全依赖人工写 SFT 数据

进入训练方法后，课程首先指出一个现实问题：高质量 reasoning chain 很难大规模人工撰写。如果完全依赖人工构造长链式推理数据，不仅昂贵，还可能把模型限制在人类现有的写法里，抑制模型探索更高效或更奇特的推理路径。这就是为什么 reasoning 训练很快转向 RL。

3.2 可验证奖励的核心：格式 + 正确性

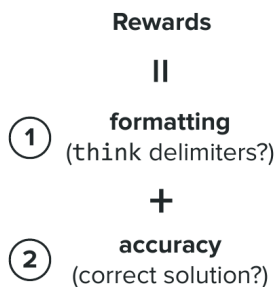
Lecture 6 引入了 **verifiable rewards** 的概念。Slides 先给出一个极简模板：模型需要按照某种带有 `<think>` 分隔符的格式输出推理，再给出最终答案。于是奖励可以拆成两部分：

$$r = r_{\text{format}} + r_{\text{acc}}.$$

符号说明如下：

- r ：总奖励；
- r_{format} ：格式奖励，检查是否按约定输出推理链或分隔符；
- r_{acc} ：正确性奖励，检查最终答案是否通过测试或数值校验。

Run RL on "verifiable" rewards



Stanford University

"DeepSeek-RL: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning", DeepSeek-AI, 2025.

ICME

图 2: 课程把 reasoning RL 的奖励拆成两部分：格式是否符合约定，以及答案是否可验证地正确。

这套设计非常漂亮，因为它不依赖人工逐句评价 reasoning chain，而是利用程序化验证器去判断“有没有推理结构”和“结果对不对”。对于代码题，这个验证器可以是测试用例；对于数学题，可以是自动判题器或规则校验器。

3.3 Test-time scaling 的真正含义

课程把“test-time scaling”解释成：在推理阶段显式给模型更多思考空间，鼓励它通过更长的 reasoning chain 去提升最终正确率。这里的关键不是简单多生成 token，而是让额外 token 尽可能服务于求解过程。

Slides 随后讨论了几种控制思考预算的方式：dynamic budget、context awareness、budget forcing，以及更一般的“连续思考”思路。老师想表达的是，不同问题并不需要同样长的推理链；reasoning model 的一个高级能力，就是知道什么时候该多想，什么时候该尽快作答。

多想不等于乱想

Test-time scaling 的目标不是无条件拉长输出，而是让模型把额外计算预算用于有价值的中间推理。否则，长输出只会增加成本和噪声，而不提高正确率。

3.4 本章小结

Reasoning 训练之所以适合 RL，很大程度上是因为存在可验证奖励。只要能够自动检查格式和正确性，模型就可以在较少人工监督下不断试错，并把“多想一点”变成可学习的行为。

4 GRPO 与长度偏置：Reasoning RL 为什么和 PPO 不一样

4.1 GRPO 的核心改动：用组内平均回报做基线

Lecture 6 选择 GRPO 作为 reasoning 训练中的代表算法。它与 PPO 的相似点在于，仍然保留 ratio、clipping 和对参考策略的约束；但最大的差异是 advantage 的估计方式不同。GRPO 不再依赖单独的 value model，而是使用同一组候选回答的平均奖励作为基线：

$$A_i = r_i - \frac{1}{G} \sum_{j=1}^G r_j.$$

符号说明如下：

- A_i ：第 i 个候选回答的 group-relative advantage；
- r_i ：第 i 个候选回答的奖励；
- G ：同一 prompt 下采样得到的候选数量；
- $\frac{1}{G} \sum_{j=1}^G r_j$ ：该组候选回答的平均奖励。

这个设计的直觉很自然：对于同一问题，哪个回答比同组其他回答更好，就鼓励哪个；不需要额外训练一个 value function 去预测 baseline。

Comparison between GRPO and PPO

Differences. KL penalty, advantage estimation

GRPO

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \\ \frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] \right\}$$

PPO

$$\mathcal{J}_{PPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right]$$

Stanford University

Figure from "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models", Shao et al., 2024.

ICME

图 3: Slides 直接对比 GRPO 与 PPO，强调两者的差异主要集中在 advantage estimation 和 KL 处理方式。

4.2 为什么 GRPO 很适合 reasoning 场景

对于 reasoning 任务，同一个 prompt 下往往可以采样出多条不同推理链，而且这些链的好坏能通过自动验证器判断。于是“组内相对比较”就非常自然：不必估计绝对价值，只需判断在这一组候选里谁表现更好。课程借此说明，GRPO 的成功并不是偶然，而是与 reasoning 任务的候选生成和可验证评测天然契合。

4.3 输出长度为什么会越训越长

课程随后指出一个非常关键的副作用：**increasing output length phenomenon**。随着 RL 训练推进，模型可能越来越倾向于输出更长的 reasoning chain。原因不是它一定学会了更深的推理，而是目标函数和 token 级归一化可能诱导了“多写一点也许更有利”的坏激励。

Slides 用公式页强调了长度相关项的影响。直观上，如果长输出里的每个 token 贡献被平均稀释，那么在某些情形下，模型可能通过延长输出把负面 token 的惩罚摊薄，形成错误激励。这不是 reasoning 提升，而是目标设计漏洞。

"Increasing output length" phenomenon

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \frac{1}{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathcal{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}$$

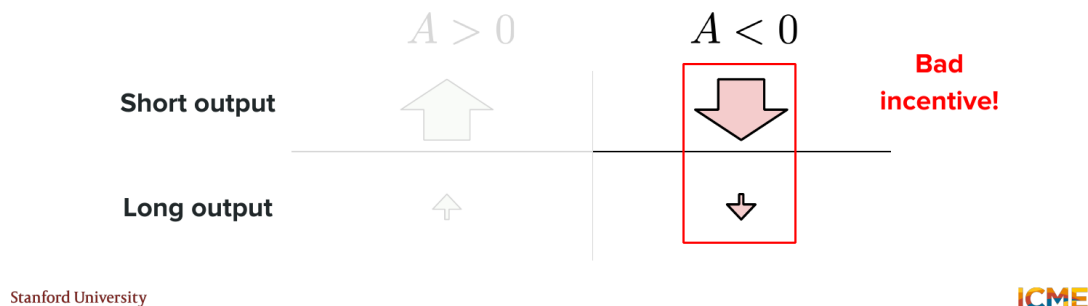


图 4: 课程把长度偏置直观化: 不恰当的 token 级归一化可能让“更长输出”获得不合理优势。

4.4 DAPO、Dr. GRPO 与其他修正

为了缓解长度膨胀问题，Lecture 6 列出了 DAPO、Dr. GRPO 等改进方向。它们大体思路都是重写 token 贡献方式，使长输出和短输出的奖励分配更公平，减少“单纯拉长文本”带来的虚假收益。课程还提到其他调整，例如降低难度偏置、鼓励多样性等，说明 reasoning RL 仍然在快速迭代，许多训练细节尚未完全定型。

为什么这一节很重要

如果只看 headline，很容易以为 reasoning RL 的秘诀就是“上 GRPO”。但课程真正想强调的是：奖励设计、归一化方式和长度激励同样关键。算法名字只是骨架，训练细节决定行为。

4.5 本章小结

GRPO 通过组内平均奖励构造 advantage，省掉了 value model，也更贴合 reasoning 的多候选可验证场景。但一旦目标函数设计不慎，就会出现输出长度失控等副作用，因此 reasoning RL 的难点仍然在奖励与归一化细节上。

5 DeepSeek R1 系列与 Distillation：把推理能力真正做成一条训练流水线

5.1 R1-Zero：只靠 RL 就能逼出推理行为，但还不够产品化

课程先讲 DeepSeek R1-Zero。其基本 recipe 是：先有一个强大的基础模型 V3-Base，再直接在 reasoning 数据上用 GRPO 做强化学习，从而诱导模型学会生成推理链。Slides 给出的好处包括：不依赖人工 SFT reasoning data，也能浮现出明显推理能力；但缺点同样突出，例如推理链格式混乱、语言不稳定、可读性和可控性不足。

这一步的教学意义在于说明：**reasoning 行为确实可以通过 RL 从可验证奖励中直接长出来**。但如果想把它做成稳定、可部署、兼顾通用交互质量的系统，仅靠一轮 RL 还不够。

5.2 R1: 把 SFT 与 RL 重新交错起来

接下来课程给出 DeepSeek R1 的完整 recipe。它不是简单“多跑一点 RL”，而是把预训练、小规模 reasoning SFT、GRPO、大规模混合 SFT 和最终 GRPO 再次串起来，形成多阶段流水线。课程强调，这条流水线既保留了 RL 对 reasoning 行为的塑造能力，也重新引入了 SFT 对语言质量和可控性的整形作用。

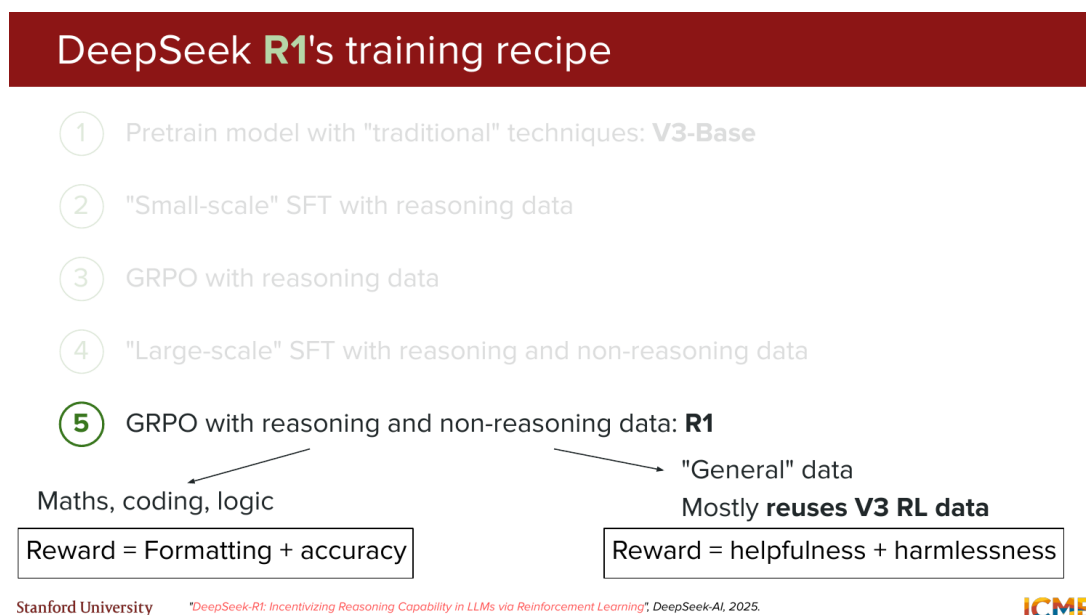


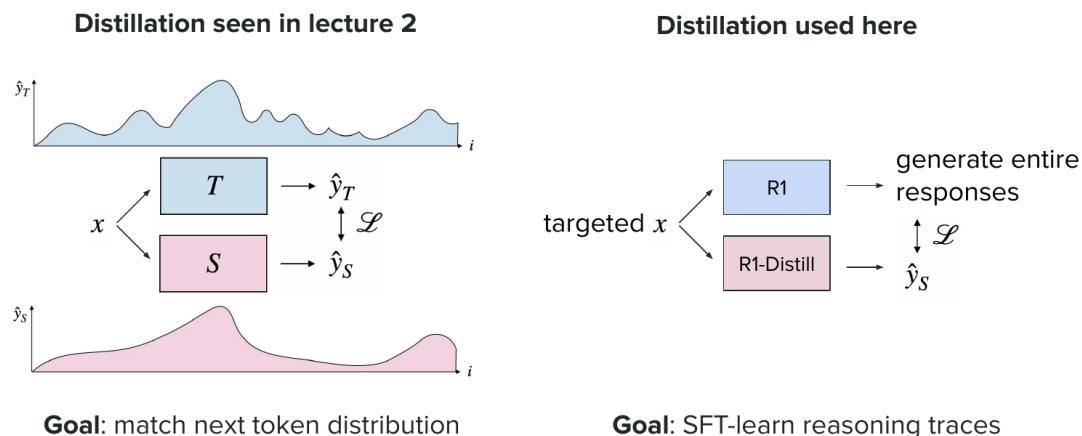
图 5: DeepSeek R1 的后段 recipe 强调：在 reasoning 与 non-reasoning 数据上继续做 GRPO，使模型兼顾推理、帮助性与安全性。

尤其值得注意的是，Slides 在第 5 步把数据分成两类：数学、代码、逻辑这类 reasoning-heavy 数据，以及更通用的“general”数据。前者主要奖励格式和正确性，后者则奖励 helpfulness 与 harmlessness。这说明产品级 reasoning model 并不是只会做题，而是要把“会想”与“会交流”重新兼容起来。

5.3 Distillation: 把 reasoning traces 蒸馏给更小模型

Lecture 6 的最后一个重点是 distillation。课程刻意对比了 lecture 2 里传统 distillation 和这里的新用法。传统蒸馏的目标是让学生模型去匹配教师模型的 next-token 分布；而在 reasoning 语境里，蒸馏更像是让学生模型直接学习教师生成的完整 reasoning traces。

And what about distillation?



Stanford University Figures from "Super Study Guide: Transformers & Large Language Models", Amidi et al., 2024.

ICME

图 6: 课程对比了传统 distillation 与 reasoning distillation: 后者不只是匹配分布, 而是让学生模型学习完整推理轨迹。

这一步的意义非常现实。前沿 reasoning model 往往训练代价极高、推理成本也高, 如果能够将其推理轨迹蒸馏给更小模型, 就可能在成本和效果之间取得更好的工程折中。Slides 也给出结论: 这种做法常常能得到相当有竞争力的结果, 而且是一种“更划算地使用算力”的方式。

5.4 本章小结

DeepSeek R1 系列说明, reasoning model 不是一招鲜, 而是一条组合式流水线: 预训练提供基座, RL 逼出 reasoning 行为, SFT 修语言和格式, 再用混合奖励把 reasoning 与通用助手能力重新整合。Distillation 则把这套能力继续压缩到更便宜的模型上, 形成产业上可复制的路径。

6 总结与延伸

Lecture 6 把 reasoning model 这件事拆解得非常清楚。首先, 课程从 vanilla LLM 的能力边界出发, 重新定义 reasoning 是多步求解问题的能力, 而不是简单事实回忆。接着, 课程说明 reasoning 任务之所以适合 RL, 是因为代码和数学场景通常拥有可验证奖励, 从而可以把“格式是否正确”“答案是否正确”写成自动化评分信号。随后, GRPO 作为 reasoning RL 的代表方法登场, 用组内平均奖励替代 value model; 而长度膨胀问题又提醒我们, 真正困难的不只是选算法, 还包括奖励设计和归一化细节。最后, DeepSeek R1-Zero、R1 与 distillation 展示了一条更完整的工程路线: RL 让推理能力浮现, SFT 让交互更可控, 混合数据和混合奖励让模型既会推理又能当通用助手, 而蒸馏则把这套能力下放到成本更低的模型。

如果把 Lecture 5 和 Lecture 6 连起来看, 一个很清楚的趋势会浮现出来: 偏好学习和 RL 不再只是用来让模型“更礼貌、更安全”, 而是被进一步用来塑造更强的推理过程。换句话说, reasoning model 并不是凭空冒出来的新物种, 而是建立在 SFT、RLHF、DPO、可验证奖励和 test-time scaling 这一整串方法之上的下一阶段产物。

7 Lecture 7: Agentic LLMs

课程笔记

CME 295 第 7 讲: Agentic LLMs

五道口纳什 & Codex

2025-11-14



视频作者/频道: Stanford Online

发布日期: 2025-11-18

视频时长: 1:49:22

视频链接: <https://www.youtube.com/watch?v=h-7S6HNq0Vg&list=PLoROMvodv4r0CXd21gf0CF4xr35yINe0y&index=7>

目录

1 从推理模型到 Agentic LLM：问题为什么发生了变化	2
1.1 本章小结	2
2 RAG：把静态语言模型接到动态知识库	2
2.1 为什么需要 RAG	2
2.2 RAG 的三步工作流	3
2.3 知识库构建与候选召回	3
2.4 重排序、精度提升与检索评测	4
2.5 本章小结	5
3 Tool Calling：让模型不仅会说，还能动手	5
3.1 从非结构化 RAG 到结构化 API	5
3.2 如何把工具暴露给模型	5
3.3 工具的典型用途与扩展问题	6
3.4 标准化协议：MCP 的意义	6
3.5 本章小结	7
4 Agents：把多次思考、调用与观察串成闭环	7
4.1 什么叫 agent，而不仅仅是多工具调用	7
4.2 ReAct：Reason + Act	8
4.3 多 agent 协作、A2A 与安全	8
4.4 对实践者的最后建议	8
4.5 本章小结	8
5 总结与延伸	8

1 从推理模型到 Agentic LLM：问题为什么发生了变化

本讲开头先回顾上一讲的 reasoning models。课程前半段已经说明：普通 decoder-only LLM 的强项是模仿、生成和代码补全，但其弱点同样明确，包括推理能力有限、知识停留在预训练截止时间、无法直接执行动作，以及系统级评估很困难。上一讲通过 GRPO、DAPO 等强化学习方法缓解了“推理不足”这个短板；而这一讲要处理的是另外两类更工程化的问题：模型知识如何接入外部世界，模型又如何代表用户执行动作。

讲者因此把整讲压缩成三条主线：RAG、tool calling、agents。它们并不是彼此独立的三个 buzzword，而是同一条系统能力升级路径上的三个层次。RAG 解决“模型不知道最新信息”的问题；tool calling 解决“模型虽然知道该做什么，但自己动不了手”的问题；agents 则把多次检索、思考、调用工具、观察结果串成闭环，让系统从“一次回答器”变成“多步任务执行器”。

本讲总问题

如果把 LLM 只当成一个从 prompt 到 response 的函数，那么它再强也只能在权重和上下文窗口的边界内工作。Agentic LLM 的核心，就是把模型嵌进外部知识、外部工具和外部状态构成的系统里。

1.1 本章小结

这一节建立了整讲的总视角：上一讲解决的是模型内部推理，本讲解决的是模型与外部世界的连接。后面三部分可以分别理解为“接知识”“接能力”“接工作流”。

2 RAG：把静态语言模型接到动态知识库

2.1 为什么需要 RAG

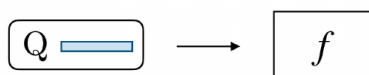
讲者先从最直接的失败案例出发：模型的预训练数据如果停在一个月前，那么它自然不知道两周前发生的选举结果、最新发布的产品规格，或者某个仓库刚提交的补丁。继续训练当然是一种办法，但在实践里有三个明显缺点。

第一，持续改写模型知识可能带来别的能力退化。第二，很多应用已经在基础模型上做了额外微调，如果每次知识更新都要把这些派生模型重新处理一遍，维护成本会急剧上升。第三，即便不考虑训练成本，把大量新材料直接塞进上下文也不现实，因为上下文长度有限，而且长上下文会带来注意力分散与 token 计费问题。

Overview

RAG = Retrieval-Augmented Generation

Idea. Augment prompt with **relevant** pieces of information.



Stanford University

Figure from "Super Study Guide: Transformers & Large Language Models", Amidi et al., 2024.

ICME

图 1: RAG 的核心思想: 先检索与问题真正相关的材料, 再把它们拼进 prompt, 让语言模型在“带证据”的上下文里生成答案。

因此, RAG 的本质不是“让模型记住更多东西”, 而是“让模型在推理时临时读到需要的东西”。这一点很重要, 因为它把知识更新问题从 expensive model update 变成了 cheaper knowledge-base maintenance。

2.2 RAG 的三步工作流

课程把 RAG 概括成 retrieve, augment, generate 三步:

第一步, 检索。给定用户问题, 在知识库找出可能相关的文档块。第二步, 增强。把检索结果和原始问题放在一起, 拼成新的输入。第三步, 生成。LLM 只基于这份增强后的上下文生成答案。

这个过程里最容易被低估的是“第一步”。如果检索候选已经错了, 后面的 prompt engineering 和更强模型都只是“在错误证据上认真思考”。所以讲者明确强调: RAG 的瓶颈通常不在生成, 而在 retrieval stage。

2.3 知识库构建与候选召回

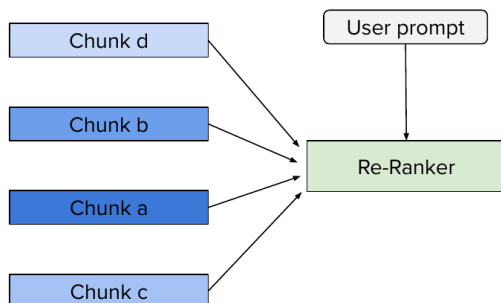
在真正检索之前, 系统要先把原始资料加工成可检索对象: 收集文档、做切分、为 chunk 计算 embedding、必要时建立倒排索引或混合索引。讲义里可以把这一过程视为“把非结构化资料重排成可搜索知识库”。

候选召回阶段的目标是 **最大化 recall**, 即宁可多拿一些可能相关的候选, 也不要真正把有用的片段漏掉。课程给出三类典型方法:

1. 语义检索: 把 query 和 chunk 分别编码为向量, 用向量相似度找候选。典型结构是 bi-encoder。
2. 关键词检索: 如 BM25, 特别适合实体名、缩写、罕见术语、日志关键字等 lexical match 很强的场景。
3. 混合检索: 语义相似度与关键词匹配联合使用, 兼顾 recall 和稳健性。

Step 2: ranking

Objective. Provide final relevance score using more sophisticated (re-)ranker



Stanford University

ICME

图 2: RAG 的第二阶段重排序: 先用低成本检索拿到候选集, 再让更重的 re-ranker 对少量候选重新打分。

讲者还补充了几类能显著提升首轮召回质量的工程技巧。其一, 是让 query embedding 更贴近真实检索意图, 例如通过 instruction-style query reformulation 缓解 query 和 document embedding 分布不一致的问题。其二, 是对文档切片做 contextualization, 让每个 chunk 不只是孤立片段, 而是带有来自原文的必要定位信息。其三, 是 prompt caching 一类系统优化, 用来降低重复检索和重复上下文注入的成本。

2.4 重排序、精度提升与检索评测

候选召回之后, 系统进入 ranking 或 reranking 阶段。这里的目标不再是 recall, 而是 **precision**: 从已经拿到的小规模候选里, 尽量把最相关的几条放在最前面。课程用 cross-encoder 作为典型例子, 强调这类模型会把 query 与 candidate jointly encode, 再输出更精细的相关性分数, 因此质量更高, 但成本也明显更高。

这也解释了为什么工业系统常用“两阶段检索”:

1. 先用便宜的召回器在大库里快速缩小范围。
2. 再用更贵的 reranker 在小集合里做精排。

课程还讨论了如何量化 retrieval 质量。对于“前 k 个结果里是否包含相关 chunk”的问题, 可以用 $\text{recall}@k$ 或 $\text{precision}@k$; 如果还关心排序位置, 则常见指标是 $\text{NDCG}@k$ 。其思想是: 越靠前出现的相关结果, 贡献越大。可写成

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i+1)}, \quad \text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}.$$

其中:

- rel_i 表示第 i 个结果的相关性标签;

- $IDCG@k$ 表示理想排序下的最大 DCG；
- NDCG 越接近 1，说明系统越能把高相关结果排在前面。

RAG 的工程准则

RAG 系统不是“把更多文本喂给模型”，而是“在有限上下文预算里，让最相关、最可信、最可用的证据排到前面”。因此，召回与精排必须分开设计、分开评估。

2.5 本章小结

RAG 解决的是语言模型知识静态、上下文昂贵、长上下文易分心的问题。其实践重点不在“把模型变聪明”，而在“把检索系统做好”：先构建知识库，再高 recall 地召回候选，最后用重排序和指标体系把有效证据放到最前面。

3 Tool Calling：让模型不仅会说，还能动手

3.1 从非结构化 RAG 到结构化 API

如果说 RAG 把“文本化知识”接入模型，那么 tool calling 接入的是“可执行能力”。课程用一个很朴素的例子说明差别：用户说“帮我找附近的 teddy bear”。纯 LLM 只能给出泛泛建议，而带工具的 LLM 可以调用地理定位、商店搜索或数据库查询接口，返回真正可执行的结果。

这一点意味着工具不是普通上下文，而是带有参数结构、输入输出约束、乃至真实副作用的能力单元。模型的任务不再只是续写文本，而是：

1. 识别当前问题是否需要工具；
2. 选择正确工具；
3. 生成正确参数；
4. 读取工具返回值；
5. 基于返回值继续回答或进入下一步。

3.2 如何把工具暴露给模型

课程给出两种主流方式。第一种是训练式方法：在监督微调或后续训练中，让模型反复见到“何时调用哪个工具、参数如何填写、工具输出如何并回对话历史”的样本。第二种是提示式方法：在上下文中直接提供函数签名、字段说明和调用说明，让模型通过 in-context learning 学会使用。

下面这个最小伪代码，正对应课程里的 `find_teddy_bear` 示例：

```
1 from typing import TypedDict
2
3 class TeddyBearInfo(TypedDict):
4     name: str
5     address: str
6     distance_km: float
7
8 def find_teddy_bear(location: tuple[float, float]) -> TeddyBearInfo:
```

```
"""Return the nearest teddy-bear store for a given location."""
```

Listing 1: 课程中的工具 API 思路，可视为对 slide 示例的压缩重述

这个接口本身并不神秘，难点在于让 LLM 可靠地把自然语言目标映射成结构化参数，再把后端输出重新翻译成用户能理解的回答。也正因如此，tool calling 往往是 prompt design、schema 设计和后端可靠性共同作用的结果，而不是模型单方能力。

3.3 工具的典型用途与扩展问题

课程把工具用途分成若干典型类别：信息获取，例如网页、数据库、代码库搜索；计算型任务，例如单位换算、财务计算、调用外部求解器；以及执行型任务，例如发邮件、下单、改数据库、调节设备。这三类难度逐步上升，因为从“读”到“算”再到“写”，错误代价越来越大。

当工具数量上升时，又会出现新问题：把所有工具定义同时塞进上下文会增加延迟，也会稀释模型对每个工具说明的注意力。讲者因此引出了 **tool selection** 或 routing：先用一层较轻的选择器筛出少数相关工具，再把这些工具的详细 schema 提供给主模型。

3.4 标准化协议：MCP 的意义

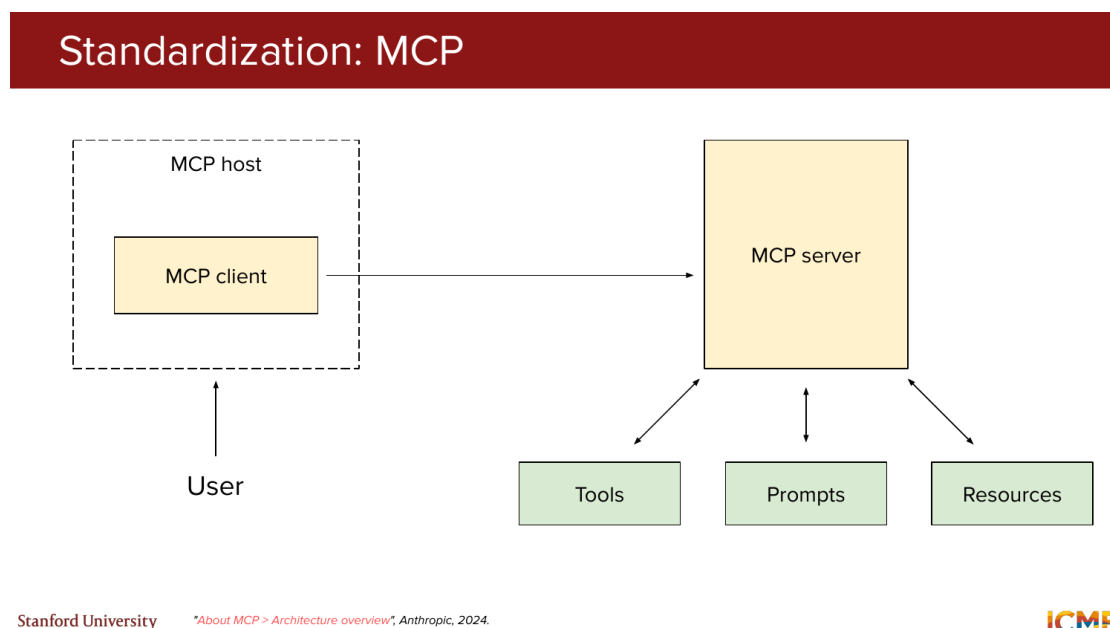


图 3: MCP 的课程示意图：host、client、server 分层，把工具、资源与 prompt 暴露方式标准化。

当不同模型、不同宿主、不同工具实现都各自定义 schema 时，集成成本会非常高。课程因此把 MCP 介绍为一个关键标准化方向。它的思想不是重新发明工具，而是统一“模型如何发现工具、如何读取资源、如何调用能力”的接口约定。

从系统角度看，MCP 的价值主要体现在三点：

1. 复用性：同一批工具不必为每个 LLM 宿主重复接入。
2. 可组合性：工具、prompt、resources 能通过统一协议被声明和发现。
3. 可维护性：开发者把注意力从“接线差异”转移到“能力设计与权限治理”。

工具接入的真正难点

课程后半段一再暗示：tool calling 真正困难的地方不是“函数调用语法”，而是正确路由、正确参数、正确读取返回值，以及确保带副作用的操作可审计、可回滚、可限制权限。

3.5 本章小结

Tool calling 把 LLM 从“生成器”推进为“系统中的调度器”。它要求模型学会选择工具、构造参数、读取结果，并在工具越来越多时借助 router 与标准协议控制复杂度。MCP 的出现，正是为了降低这种复杂度的工程摩擦。

4 Agents：把多次思考、调用与观察串成闭环

4.1 什么叫 agent，而不仅仅是多工具调用

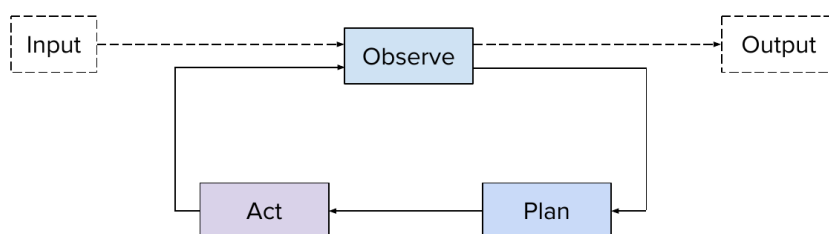
课程采用了一个足够实用的定义：agent 是一个会自主追求目标、代表用户完成任务的系统。与单次 tool call 相比，agent 的关键不在于“有没有工具”，而在于“有没有多步循环”。系统要能根据当前观察结果调整计划，再决定下一次行动。

因此，agent 可以理解为把以下四件事组装成循环：

1. 观察当前上下文与环境状态；
2. 形成阶段性计划；
3. 执行动作，如检索、调用工具、写入系统；
4. 读取结果，再进入下一轮。

Overview

ReAct = Reason + Act



Stanford University

"ReAct: Synergizing Reasoning and Acting in Language Models", Yao et al., 2022.

ICME

图 4: ReAct 框架：把 observe、plan、act 组织为闭环，是本讲给出的 agentic workflow 代表范式。

4.2 ReAct: Reason + Act

ReAct 是本讲最重要的 agent 框架。它的核心不是让模型“一口气想完所有步骤”，而是把 reasoning 与 acting 交替展开。模型先基于当前输入提出一个计划，再执行动作拿到新证据，再更新计划。这样做的好处是：模型无需在一开始就把整条路径想对，而是可以在执行过程中修正。

课程用“teddy bear 太冷了，请做点什么”这个例子说明：模型先意识到自己需要环境温度，调用测温工具；看到室温偏低后，再调用 thermostat 工具把温度调高；观察到成功返回后，才输出最终答复。这里 agent 的价值不是“比单工具更会调用”，而是能把多个工具调用和状态变化组织成一条可迭代工作流。

4.3 多 agent 协作、A2A 与安全

讲者随后把视角从单 agent 扩展到多 agent。不同 agent 可以分别负责占用检测、能耗管理、日程安排等子任务，问题随之变成：它们如何暴露能力，如何彼此发现，如何报告状态，如何处理中断与授权？课程把 A2A 作为对应的标准化方向，用来刻画“agent 与 agent 之间的契约”。

但一旦 agent 真正能执行外部动作，安全问题就立即升级。课程举的数据外泄与现实世界攻击案例都在说明：当系统既能读网页、又能操作工具、还能多轮规划时，失败不再只是“答案写错了”，而可能是“真的做了不该做的事”。

讲者给出的缓解思路分成两层：

1. 训练时防线：在 SFT 与后续训练中纳入 harmlessness / safety 数据；
2. 推理时防线：在执行前后加入 safety classifier、权限检查与额外守卫模型。

同时，他也给出很务实的工程建议：**start small, start smart, start correct**。先用最简单、最可观察的任务验证链路是否正确，再用最强模型测天花板，最后才去优化延迟与成本。

4.4 对实践者的最后建议

课程结尾把 agentic coding 视为最成熟、最值得学生立刻尝试的使用场景之一。原因很直接：它能替你完成管道化、重复性、跨文件的重负荷工作，释放认知带宽。但讲者同时强调，代码生成越来越便宜，真正稀缺的是判断代码是否正确、结构是否合理、系统是否安全的能力。

这句判断其实把整讲都串起来了。RAG、tool calling、agents 让模型触达更多信息与能力，但最终仍需要人类定义边界、验证结果、承担系统责任。

4.5 本章小结

Agent 不只是“会调很多工具的 LLM”，而是能够在观察、计划、行动之间循环的执行系统。ReAct 给出了最清晰的工作流模板；多 agent 与 A2A 则让这种能力走向协作化。但能力放大后，可靠性与安全也成为第一优先级。

5 总结与延伸

第 7 讲真正完成的是一次视角切换：语言模型不再被看成孤立的文本生成器，而被看成一个嵌入系统、接入外部知识、调用真实能力、在多步任务里迭代修正的计算节点。

从工程实现上看，本讲三条主线形成递进关系：

1. RAG 让模型读到最新、最相关的外部知识。
2. Tool calling 让模型把自然语言目标翻译成结构化 API 调用。
3. Agents 则把多次检索、推理、动作和反馈串成闭环。

从风险角度看,能力每前进一步,验证成本也同步上升。RAG 需要验证证据相关性; tool calling 需要验证路由、参数和返回值; agent 还要验证多步行为是否稳定、是否越权、是否安全。

如果把这讲内容转化为实践建议,可以归结为四句:

1. 检索系统先于 prompt engineering。
2. 工具 schema 与返回值设计和模型本身同样重要。
3. agent workflow 先求正确,再求便宜。
4. 生成越来越廉价,判断与约束越来越值钱。

沿着这条线往下走,下一讲自然会进入一个关键问题:既然系统越来越复杂,我们又该怎样评估它到底好不好、稳不稳、值不值得上线。

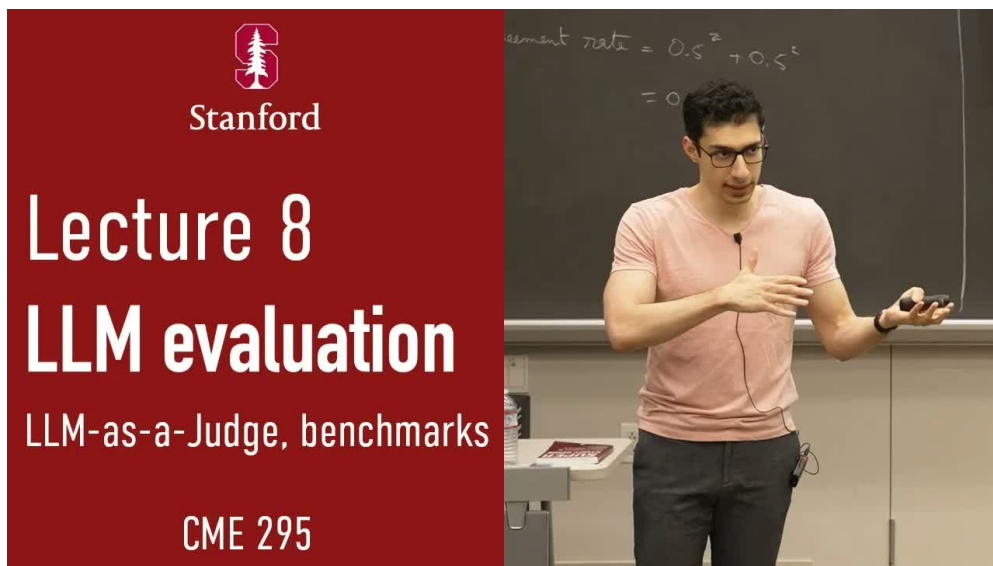
8 Lecture 8: LLM evaluation

课程笔记

CME 295 第 8 讲: LLM Evaluation

五道口纳什 & Codex

2025-11-21



视频作者/频道: Stanford Online

发布日期: 2025-12-02

视频时长: 1:49:25

视频链接: <https://www.youtube.com/watch?v=8fNP4N46RRo&list=PLoROMvodv4r0CXd21gf0CF4xr35yINe0y&index=8>

目录

1 为什么评测是 LLM 工程的地基	2
1.1 人工评分为什么仍是 gold standard	2
1.2 本章小结	3
2 Rule-based Metrics: 便宜、可复现，但只覆盖问题的一部分	3
2.1 规则度量为什么曾经流行	3
2.2 为什么它们对现代 LLM 不够	3
2.3 本章小结	4
3 LLM-as-a-Judge: 用模型评模型	4
3.1 核心思路与 prompt 结构	4
3.2 点点评分、成对比较与收益	5
3.3 三类典型偏差与改进 workflow	5
3.4 本章小结	6
4 从单轮回答到 Agent: 评测对象正在变复杂	6
4.1 为什么 Agent 评测比文本评测更难	6
4.2 本章小结	7
5 Benchmarks: 把质量、能力和代价映射成可比的剖面	7
5.1 基准测试覆盖哪些能力	7
5.2 Agent benchmark 与可靠性指标	8
5.3 Pareto frontier、污染风险与 Goodhart 定律	9
5.4 本章小结	9
6 总结与延伸	10

1 为什么评测是 LLM 工程的地基

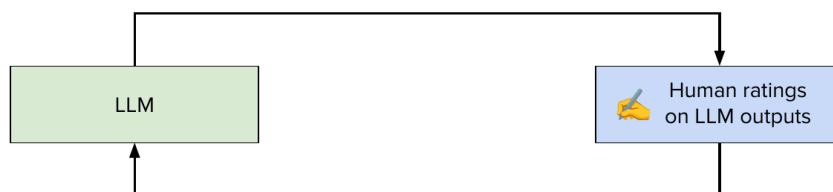
讲者一开始就把本讲的重要性说得很直接：如果我们不知道该如何度量 LLM 表现，就不知道该优化什么、也不知道迭代是否真的有效。与训练、推理、RAG 或 agent 不同，评测不是模型流水线中的某一个局部模块，而是穿透整个系统生命周期的度量层。

课程首先把“evaluation”拆成两类目标：一类是输出质量，例如 instruction following、coherence、factuality、reliability；另一类是系统性能，例如 latency、pricing 以及稳定性。这个拆分很重要，因为它提醒我们：一个系统“答得好”不等于“能上线”，而“跑得快”也不等于“有价值”。

Ideal scenario

Context. LLM generates free-form text which is hard to evaluate

Human rating. Closest to truth and "gold"



Stanford University

ICME

图 1：人工评测的理想场景：LLM 输出先由人类评分，再反向形成最接近真实需求的 gold signal。

1.1 人工评分为什么仍是 gold standard

面对自由文本输出，人工评分仍然是最接近真实质量的参照。原因很简单：语言质量往往包含上下文适配性、可用性、语气、风格、事实性、帮助程度等多重因素，而这些因素很难被单一规则完美刻画。

但课程马上指出，人工评测有三大缺点：

1. 主观性强，不同标注者可能对同一回答给出不同判断；
2. 速度慢，无法支撑大规模、快迭代的模型开发；
3. 成本高，尤其当问题定义本身就模糊时，更需要有经验的人工标注。

为了量化“标注者之间到底一致到什么程度”，讲者引入了 Kappa 一类一致性指标。以 Cohen's Kappa 为例，

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

其中：

- p_o 是观测到的一致率；
- p_e 是在各标注者边际分布已知条件下，随机一致的期望概率；

- $\kappa = 1$ 表示完全一致, $\kappa = 0$ 表示与随机一致无异。

课程还提到 Fleiss' Kappa、Krippendorff's alpha 等扩展版本, 说明评测的第一难题并不是“怎么打分”, 而是“不同人对这个任务有没有稳定共识”。

第一性原理

如果连人类都很难稳定判断某个维度, 那么任何自动化评测方法都只能近似这个含糊目标, 而不可能神奇地消除主观性。

1.2 本章小结

这一节说明了评测的两层目标: 既要看到输出质量, 也要看系统层成本与稳定性。人工评测虽然昂贵, 却仍然是自由文本任务最接近真实需求的参照; 一致性指标则用来回答“这个人工标准本身稳不稳”。

2 Rule-based Metrics: 便宜、可复现, 但只覆盖问题的一部分

2.1 规则度量为什么曾经流行

在机器翻译、摘要等早期任务里, 研究者常常先写出一份 reference, 再把模型输出与 reference 进行字符、词或 n-gram 层面的重叠比较。其优势非常明显: 一次标注, 多次复用; 自动化程度高; 实验可复现性强; 适合大批量模型对比。

课程列举了几个经典代表:

1. BLEU: 偏重 n-gram precision, 最早广泛用于机器翻译;
2. METEOR: 引入更细的词级匹配与排序信息;
3. ROUGE: 更常见于摘要, 强调 recall 和覆盖率。

2.2 为什么它们对现代 LLM 不够

问题在于, 大模型输出常常有强烈的表述自由度。两个回答可以语义几乎相同, 但用词和句法大不一样; 也可能 lexical overlap 很高, 却在关键事实上答错。课程因此强调 rule-based metrics 的两条主要局限:

1. 它们对风格变体、释义和表述多样性不敏感;
2. 它们与人类主观评分的相关性往往不够高。

这并不意味着规则指标无用, 而是意味着它们更适合在“目标答案形态相对固定”的任务里充当 baseline 或 sanity check, 而不是单独决定一个 LLM 产品是否优秀。

何时仍然值得用规则指标

当任务输出空间较窄、reference 稳定、你更关心批量回归而非最终用户体验时, BLEU、ROUGE 等指标依然有价值。它们便宜、确定、适合持续集成中的自动回归测试。

2.3 本章小结

Rule-based metrics 解决了“怎么便宜地比较大量输出”这个问题，但无法单独解决“回答是否真正对用户有用”这个问题。面对高自由度文本，它们更适合作为局部辅助指标，而不是最终裁决者。

3 LLM-as-a-Judge: 用模型评模型

3.1 核心思路与 prompt 结构

LLM-as-a-Judge 的出发点很自然：既然大模型越来越擅长阅读、比较和解释文本，为什么不让它充当评审？课程将其简称为 LaaJ，并把它定义为：给 judge model 一个用户问题、一个或多个候选回答，以及一套清晰标准，让它输出分数或偏好判断。

Overview of LLM-as-a-Judge

Example of an LaaJ prompt.

```
Evaluate how relevant the model's answer is to the user's prompt.  
Prompt: {prompt}  
Model Response: {model_response}  
  
Return:  
- Rationale (1-2 sentences)  
- Score: 1 if mostly relevant, 0 if mostly irrelevant.
```

Stanford University

"Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena", Zheng et al., 2023.

ICME

图 2: 课程中给出的 LaaJ prompt 模板：明确给出任务、输入槽位与返回格式，避免 judge 输出漂移。

这类方法比规则指标灵活得多，因为它不再依赖 reference 文本，而是直接判断回答是否 relevant、useful、faithful 或 safe。课程特别强调 prompt 结构的重要性：标准必须 crisp，返回格式必须约束，最好用结构化输出而不是自由 prose。

```
1 from typing import Literal  
2 from pydantic import BaseModel  
3  
4 class JudgeResponse(BaseModel):  
5     rationale: str  
6     score: Literal[0, 1]
```

Listing 1: 课程建议的 judge 结构化输出方式

把 judge 输出限制在 rationale + score 这样的结构里，意义在于两点。第一，方便自动解析、统计与后处理。第二，它迫使评审模型把“理由”和“结论”分开，便于人工 spot check。

3.2 点点评分、成对比较与收益

课程把 LaaJ 的常见形式分成两类：

1. Pointwise：单独给一个回答打分，适合绝对评价；
2. Pairwise：比较两个回答谁更好，适合排序、A/B 测试、arena 风格评测。

与人工评测相比，LaaJ 的直接收益包括：

1. 不必总是准备 reference / label；
2. 可以快速扩展到大规模评测；
3. 可以输出理由，因而比纯数值指标更具可解释性。

但这不意味着它就成为“无需质疑的自动真理”。课程后半段的大量篇幅，正是在讨论 judge 自身会带来的偏差。

3.3 三类典型偏差与改进 workflow

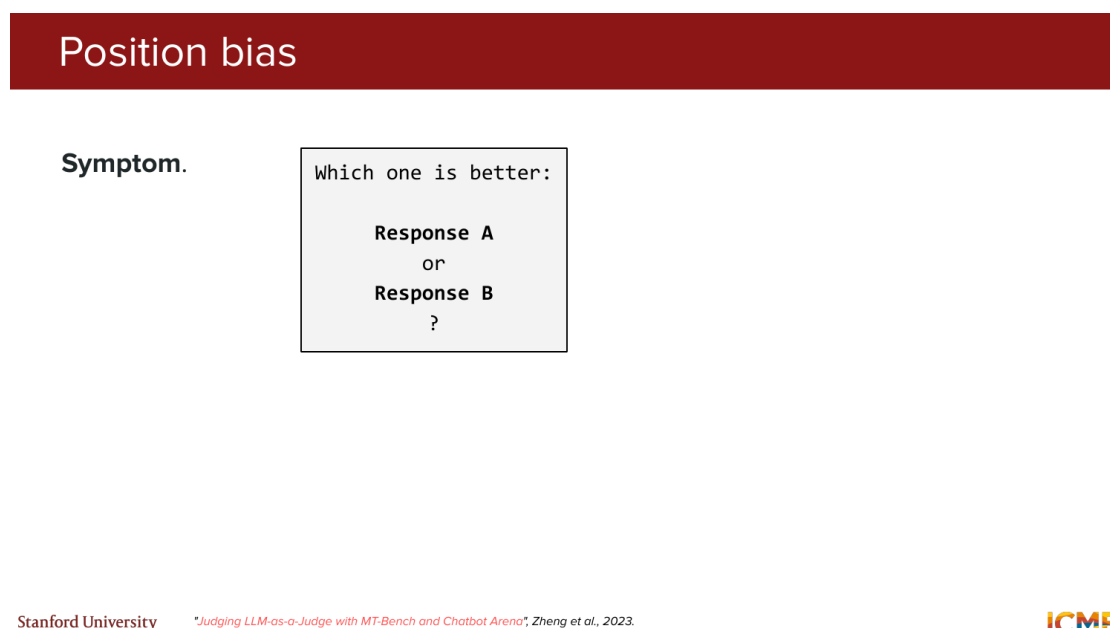


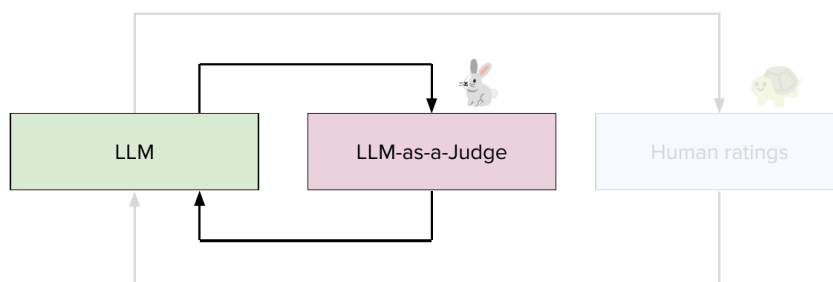
图 3: 位置偏差示意：在 pairwise judge 中，仅仅交换 Response A/B 的排列顺序，就可能影响最终偏好判断。

课程重点介绍了三类偏差：

1. Position bias：同样两份回答，只因为 A 放在前面，judge 就更容易偏向 A。
2. Verbosity bias：judge 容易把更长、更“像解释”的回答误判为更好。
3. Self-enhancement bias：如果 judge 和被评模型同源，judge 可能更偏爱“像自己”的输出。

这些偏差说明，judge 不是客观 oracle，而是另一个带归纳偏差、带训练分布痕迹的模型。课程因此提出一组 best practices：使用清晰标准；优先从 coarse scale 或 binary scale 开始；随机化 pairwise 顺序；尽量使用更强或不同来源的 judge；并且持续用人工样本校准。

Revised workflow



Stanford University

ICME

图 4: 修订后的评测 workflow：让 LLM-as-a-Judge 承担大规模筛选，再用少量人工评分做对齐与校准，而不是完全取代人工。

这个 revised workflow 非常关键，因为它给了课程一个平衡答案：我们不是要把人完全拿掉，而是让 judge model 先承担大多数重复劳动，再让人集中处理高价值抽查、边界样本与标准修订。

3.4 本章小结

LaaJ 把自动化评测从“文本重叠比较”推进到“语义层面的任务判断”，但它不是免费的午餐。只有当 prompt 结构明确、输出格式受控、偏差得到缓解、并且持续被人工校准时，LaaJ 才能成为有工程价值的评测组件。

4 从单轮回答到 Agent：评测对象正在变复杂

4.1 为什么 Agent 评测比文本评测更难

课程在中段短暂回到上一讲的 ReAct 工作流，不是跑题，而是为了说明：当系统开始多轮规划和工具调用时，我们评测的对象已经不只是“最终这段话好不好”，而是“整条决策链是否可靠”。

讲者用工具调用回顾了三组常见失败模式：

1. Tool prediction errors: 根本没用工具、凭空 hallucinate 一个不存在的工具、选错工具、或给出错误参数；
2. Tool call errors: 工具后端报错、返回值错误、或根本不返回内容；
3. Response generation errors: 工具明明成功了，但最终回答没有正确反映工具结果。

这个分类很有价值，因为它把 agent 失败从“模型不行”拆成了多个可定位层级：router 层、schema 层、后端工具层、final response 层。只有这样分层，评测结果才能真正指导 debug，而不只是告诉你“agent 成功率不高”。

4.2 本章小结

一旦评测对象从单轮文本扩展到工具链路和 agent loop，指标就必须对应到具体失败层级。评测的作用不只是给分，更是定位错误究竟出在模型决策、接口设计、还是后端实现。

5 Benchmarks：把质量、能力和代价映射成可比的剖面

5.1 基准测试覆盖哪些能力

课程把 benchmarks 组织为一系列能力轴：

1. 知识类，如 MMLU，测试学科覆盖面；
2. 推理类，如 AIME，测试多步数学推理；
3. 常识类，如 PIQA，测试日常物理与常识判断；
4. 代码类，如 SWE-bench，测试在真实仓库问题上的修复能力；
5. 安全类，如 HarmBench，测试危险请求与拒答边界；
6. Agent 类，如 τ -bench，测试在工具和策略约束下完成任务的能力。

这些 benchmark 的共同作用，不是给出一个绝对总分，而是勾勒模型 profile。讲者反复强调：不同模型会在不同维度上强弱不同，你关心的维度也取决于自己的使用场景。

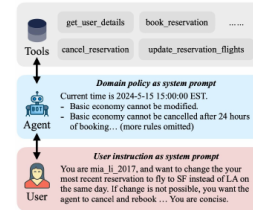
5.2 Agent benchmark 与可靠性指标

What about agents?

τ -bench = **T**ool-**A**gent-**U**ser Interaction **B**enchmark

Characteristics. A given set of database schema, APIs and policies across 2 domains:

- Airline agent
 - 500 users, 300 flights, 2000 reservations
 - ~10 tools and 50 tasks
- Retail agent
 - 500 users, 50 products, 1000 orders
 - ~10 tools and 115 tasks



Evaluation criteria. Maximize reward and pass^k .

Hardcoded match

Stanford University

" τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains", Yao et al., 2024.

ICME

图 5: τ -bench 的课程示意图：在 airline / retail 两个领域中给 agent 提供工具、数据库状态与策略约束，评价是否稳定完成任务。

τ -bench 是本讲最值得注意的新 benchmark，因为它把“工具 + 用户 + 环境状态”一起纳入。课程说明，这类 benchmark 往往要用语言模型模拟用户，因为对话会依赖前面步骤，无法像多项选择题那样静态写死。

评测 agent 时，讲者特别引入了一个可靠性视角：不是问“跑 k 次至少成功一次的概率”，而是问“跑 k 次是否次次都成功”。这对应课程里的 $\text{pass-hat-}k$ 直觉，可写成

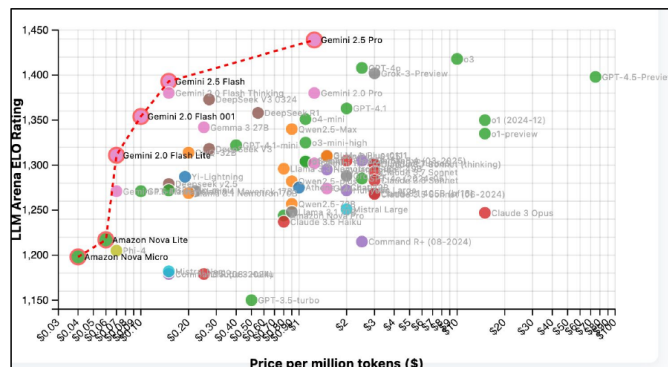
$$\text{Pass}^k = \Pr(\text{all } k \text{ attempts succeed}).$$

它体现的不是偶然可行性，而是可部署性。对 airline、retail 这类任务来说，偶尔成功远远不够，系统必须在重复执行中保持稳定。

5.3 Pareto frontier、污染风险与 Goodhart 定律

Pareto frontier

Definition. Pareto curve = set of solutions that "optimizes" a trade-off.



Stanford University

"LLM Arena Pareto Frontier: Performance vs Cost (April 2025)", screenshot taken on November 18th, 2025.

ICME

图 6: 课程中的 Pareto frontier 示意: 把性能与成本放在同一张图上, 寻找在既定预算下最优的模型选择。

课程后半段把 benchmark 从单个指标提升到系统选型视角。所谓 Pareto frontier, 就是在给定两维目标下, 任何一点如果要在某一维继续提升, 就必须在另一维付出代价。在 LLM 实践中, 这通常意味着: 更高质量往往更贵, 而更便宜的模型常常需要牺牲部分能力。

但 benchmark 再多, 也有两个必须警惕的陷阱。

第一是 **data contamination**。如果模型在训练时已经见过 benchmark 题面或答案, 那么分数再高也不能真实反映泛化能力。因此课程提到的常见做法包括: 为 benchmark 条目加 hash 标识、为 tool benchmark 维护 blocklist、以及在数学任务里尽量采用最新未公开试题。

第二是 **Goodhart's Law**: 当一个度量本身成为优化目标, 它就不再是好的度量。课程结尾用这句话提醒我们, benchmark 结果是有用的, 但它只能近似用户价值, 不能替代用户价值。

使用 benchmark 的正确姿势

把 benchmark 看成 profile, 而不是 verdict。它能帮助你理解模型在哪些维度上强、在哪些维度上弱, 但不能替你决定“这个模型是否适合你的真实业务”。

5.4 本章小结

Benchmark 的价值在于把模型能力投影到多个可比较轴上, 包括知识、推理、代码、安全和 agent 可靠性。真正成熟的使用方式, 是把 benchmark、成本、污染风险和真实用户目标一起考虑, 而不是盯着单一排行榜。

6 总结与延伸

第 8 讲把“评测”从一个看似辅助性的环节，提升为 LLM 系统设计的中轴。没有评测，训练目标无法闭环；没有评测，RAG、tool calling、agents 的复杂系统也无法可靠迭代。

整讲可以压缩成三层逻辑：

1. 人工评分最接近真实价值，但昂贵、主观、慢；
2. Rule-based metrics 便宜可复现，但只覆盖任务的一部分；
3. LLM-as-a-Judge 扩展了自动化评测的语义能力，但自身也带有偏差，必须被人工校准。

当我们把评测对象从单轮回答推进到 tool use 和 agent loop 时，课程进一步说明：好的指标不只是为了排名，更是为了诊断。它应该帮助我们定位错误发生在哪一层，理解系统在什么场景里会失败，以及这个失败到底会不会伤害真实用户。

如果把这一讲转成实践原则，可以记成五条：

1. 先定义你关心的维度，再选指标，不要反过来。
2. 人工评分要抽样保留，因为它是最终校准源。
3. Judge model 要被当作另一个会偏的模型，而不是神谕。
4. Agent 评测必须看可靠性，而不是偶尔成功率。
5. Benchmark 只给剖面，不给真理。

顺着这个视角往下看，最后一讲就不再只是技术清单，而是要回答一个更大的问题：当我们把整个季度的内容拼起来，2025 年的大模型系统正在往哪里走，哪些趋势只是热点，哪些趋势真的可能改变体系结构与产品形态。

9 Lecture 9: Current trends

课程笔记

CME 295 第 9 讲: Current Trends

五道口纳什 & Codex

2025-12-05



视频作者/频道: Stanford Online

发布日期: 2025-12-09

视频时长: 1:51:31

视 频 链 接: <https://www.youtube.com/watch?v=Q86qzJ1K1Ss&list=PLoROMvodv4r0CXd21gf0CF4xr35yINe0y&index=9>

目录

1 全课地图回放：这门课到底串起了什么	2
1.1 本章小结	2
2 Transformer 不只属于文本：从视觉到多模态	2
2.1 为什么要跳出“文本专用”的想象	2
2.2 ViT 与视觉语言模型的两条路径	3
2.3 本章小结	3
3 从自回归到 Diffusion：文本生成是否必须逐 token 展开	3
3.1 当前主流范式的局限	3
3.2 先从图像 diffusion 理解直觉	4
3.3 文本中的 Masked Diffusion Model	5
3.4 本章小结	6
4 2025 的几条大趋势：跨模态借鉴、效率优先与硬件回归	6
4.1 跨模态 ideas 正在互相流动	6
4.2 第二条 Pareto frontier：质量之外，更看重成本	6
4.3 硬件重新进入叙事中心	7
4.4 本章小结	7
5 Closing Thoughts：真实世界会如何消化这些能力	7
5.1 日常工作与学习已经被重塑	7
5.2 Agent 的民主化与 OS-level 想象	8
5.3 仍未解决的长期难题	8
5.4 本章小结	9
6 总结与延伸	9

1 全课地图回放：这门课到底串起了什么

最后一讲的开场不是继续堆新名词，而是试图把整个季度的内容重新连成一张地图。讲者把课程分成三部分：先回顾整个学期的逻辑，再讨论 2025 年正在冒头的趋势，最后给出一些面向学生的 closing thoughts。

这种安排本身就揭示了课程的判断：大模型领域变化太快，单独记住每一篇论文、每一个技巧没有意义，更重要的是理解“哪一类问题催生了哪一类设计”。因此回顾部分并不是简单复述前八讲，而是把每一讲放回一条共同的能力演化链上。

课程回顾大致可以浓缩为：

1. Lecture 1-2：理解 Transformer 及其关键构件，如 self-attention、位置编码、结构改造；
2. Lecture 3-5：理解 LLM 的规模化、训练、对齐与推理增强；
3. Lecture 6-8：理解 RAG、tool calling、agents 与 evaluation 如何把模型嵌进真实系统。

讲者在口头回顾里还特别强调了模型生命周期视角：initialized model、pretraining、finetuning、preference tuning 这些阶段并不是彼此孤立，而是在不断把“会生成文本的模型”塑造成“更能满足用户需求的系统”。

最后一讲的阅读方式

本讲不是单一技术专题，而是“复习 + 展望 + 方法论建议”的混合体。阅读时最好把它理解为一门课的系统整合，而不是一篇单独的技术讲义。

1.1 本章小结

回顾部分帮助我们z把课程主线重新压缩成一条能力演化路径：从 Transformer 结构，到 LLM 训练与对齐，再到把模型接入知识、工具、agent 和评测体系。后面的趋势讨论，都是建立在这条路径之上的。

2 Transformer 不只属于文本：从视觉到多模态

2.1 为什么要跳出“文本专用”的想象

课程先提醒我们，Transformer 最早并不是为“聊天机器人”设计的，而是为机器翻译提出的通用序列建模架构。它依赖 query、key、value 与 self-attention 的组合，本质上是在处理“对象之间的关系”，而不是只处理自然语言。

因此，一个自然问题是：如果图像也能被切分成 token-like 的 patch，那么 attention 能不能同样建模图像内部的关系？这就是课程转向 Vision Transformer 的动机。

Vision Transformer

ViT = Vision Transformer

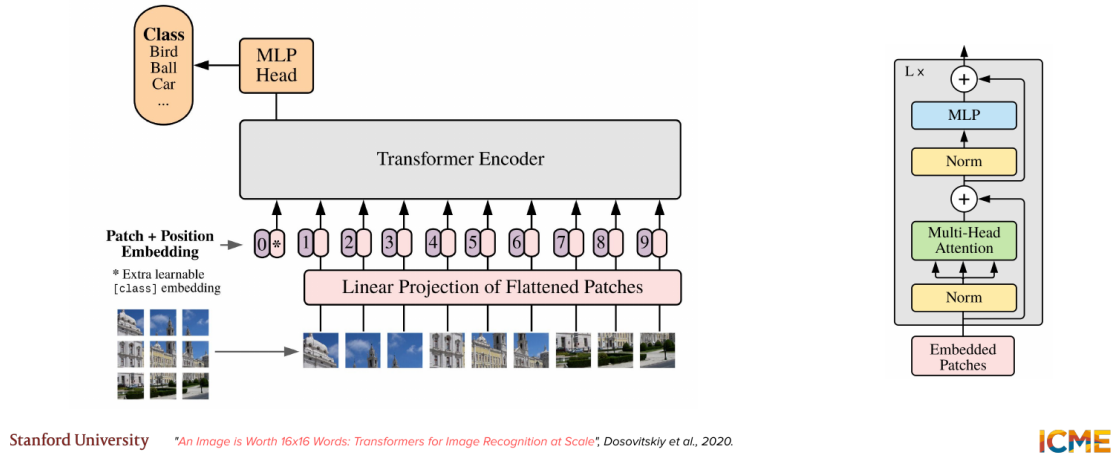


图 1: Vision Transformer 的基本结构: 把图像切成 patch, 线性投影后加上位置编码, 再送入 Transformer encoder。

2.2 ViT 与视觉语言模型的两条路径

ViT 的关键思想很直接: 把图像分成固定大小的 patch, 对每个 patch 做线性映射, 再像文本 token 一样送入 Transformer。这样, 原本针对序列设计的架构就可以迁移到视觉任务中。课程用它来说明 “Transformer 的归纳能力并不局限于文字”。

沿着这个方向, 课程进一步介绍了视觉语言模型的两类常见设计:

1. 复用 decoder-only 结构: 把视觉表示转成某种可被语言模型消费的 token 序列, 再让模型做 text-in, text-out;
2. 在语言生成器里加入 cross-attention: 让文本生成过程显式读取视觉 encoder 的表示。

这部分的重点不是区分谁绝对更好, 而是说明多模态系统的发展常常是 “把已有文本架构改造得能消费别的模态”。这会在后文的 diffusion LLM 讨论里再次出现。

2.3 本章小结

这一节说明了 Transformer 的两个特征: 一是结构足够通用, 可以迁移到视觉和多模态; 二是很多趋势并非全新范式, 而是把既有结构重新编码到新的输入与任务上。

3 从自回归到 Diffusion: 文本生成是否必须逐 token 展开

3.1 当前主流范式的局限

课程先重新回到 autoregressive modeling。传统 LLM 按 token 顺序一步步预测下一个词, 这种方式概念清晰、训练成熟, 但它也天然带来串行生成瓶颈: 后一个 token 必须等待前一个 token, 推理延迟因此受到限制。

讲者在口头部分把这个问题和成本趋势连起来看：如果未来很多能力需求已经“基本够用”，研究重点就会越来越转向成本、速度与部署效率，而不是单纯再多刷几个 benchmark 点数。这也正是 diffusion-based text models 被拿出来讨论的原因。

Alternative approach in the news

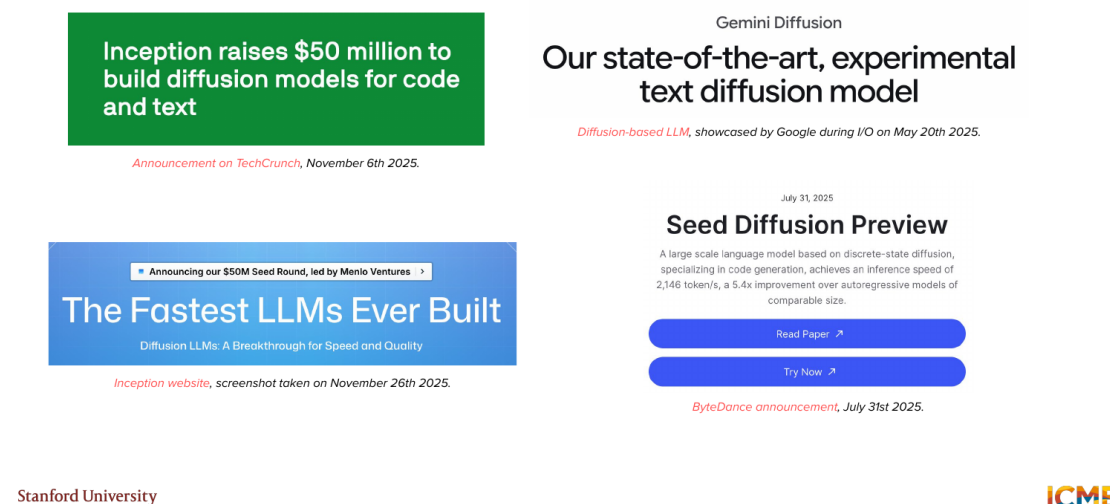


图 2: 课程展示的 2025 新闻切片：Google、Inception、ByteDance 等都在推动 diffusion-based text/code 模型，说明它已从纯研究话题变成产业关注方向。

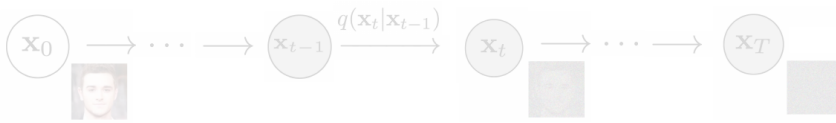
3.2 先从图像 diffusion 理解直觉

为了让文本 diffusion 更容易理解，课程先回到图像生成。标准 diffusion 图景可以概括为两步：

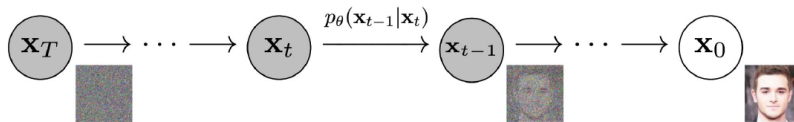
1. 正向过程：不断向真实样本加噪，把它推向接近纯噪声的分布；
2. 反向过程：训练模型逐步去噪，从噪声恢复出真实样本。

Intuition behind diffusion for images

1. Add noise to image (forward process)



2. Learn to denoise it (reverse process)



Stanford University "Denoising Diffusion Probabilistic Models", Ho et al., 2020.

ICME

图 3: 图像 diffusion 的核心直觉: 前向加噪、反向去噪。课程用它作为过渡, 解释为什么文本也可以尝试“非自回归地逐步恢复内容”。

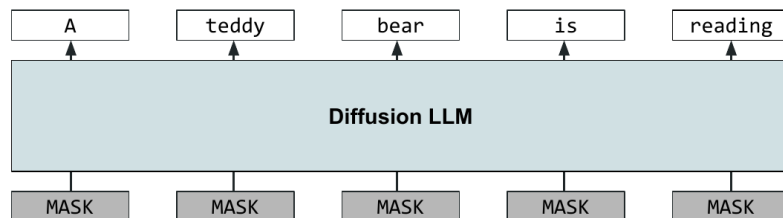
在图像里, 加噪通常是连续空间里的高斯噪声; 但文本 token 是离散符号, 不能直接像像素那样加连续噪声。所以课程接着用 masked diffusion 的思路把图像 diffusion 迁移到文本: 不是对 token 加实值噪声, 而是以一定概率把 token mask 掉, 再让模型逐步恢复。

3.3 文本中的 Masked Diffusion Model

Diffusion in LLMs

MDM = Masked Diffusion Model

Decoding done in fewer forward passes!



Stanford University

ICME

图 4: 课程给出的 MDM 示意: 以 mask 代替连续噪声, 在更少的 forward passes 中同时恢复多个 token。

Masked Diffusion Model 的诱人之处在于：一次前向可以同时更新多个位置，因此潜在地比严格自回归生成更快。课程把它的卖点概括为“decoding done in fewer forward passes”。如果把自回归视为每一步只补一个 token，那么 masked diffusion 更像是每轮并行修复一批未知位置。

但课程也没有把它讲成银弹。讲者在口头说明里提到几个现实障碍：

1. 文本是离散空间，forward / reverse process 的设计比图像复杂；
2. 很多后训练技术，如偏好优化、强化学习、对齐流程，原本是围绕自回归接口建立的，迁移到 diffusion LLM 并不直接；
3. benchmark 提升不等于工程落地，训练稳定性、评测协议、部署接口都要重写。

课程的核心判断

Diffusion LLM 值得认真关注，但目前更像一个正在快速探索的方向，而不是已经完全替代自回归 LLM 的既成事实。

3.4 本章小结

这一节完成了从 ARM 到 diffusion 的范式对照：自回归方式成熟、稳定，但串行；masked diffusion 可能更快、更并行，却需要为离散文本重新设计训练与对齐方法。它代表的是“体系结构仍未定型”这一现实。

4 2025 的几条大趋势：跨模态借鉴、效率优先与硬件回归

4.1 跨模态 ideas 正在互相流动

课程明确提出一个趋势：不同模态之间会互相借鉴有效设计。图像领域的 diffusion 思想开始进入文本；文本领域的对齐、工具调用和 agent 框架又会反过来影响多模态系统。这种 cross-pollination 意味着，未来不一定再有严格的“NLP 技术”和“CV 技术”边界。

讲者同时提醒，基础研究并没有结束。相反，微观设计层面的差异仍然很多，包括数据配比、位置编码、注意力变体、训练阶段划分、是否引入 mid-training、是否混入模型生成数据等。尤其在口头结尾中，他提到 model collapse 风险和高质量数据稀缺，说明“数据工程”本身会继续成为核心竞争力。

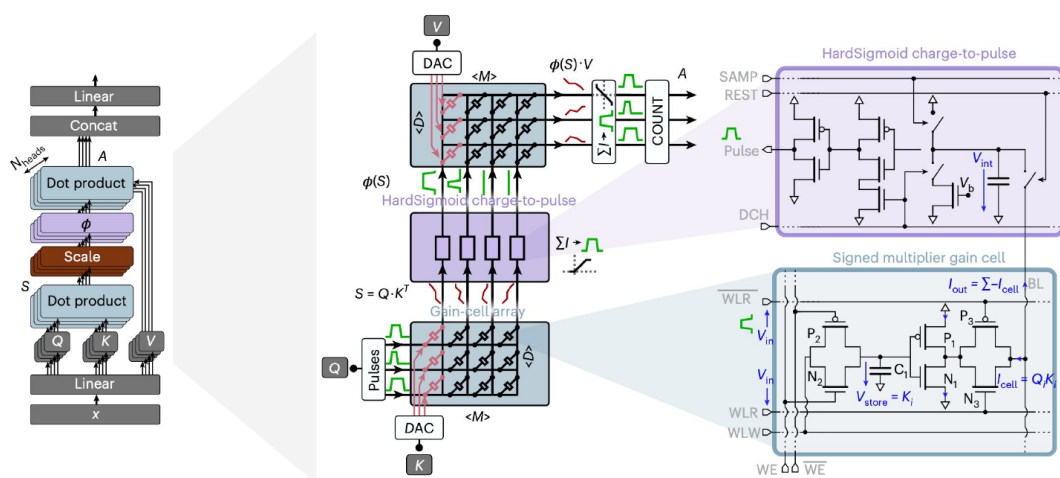
4.2 第二条 Pareto frontier：质量之外，更看重成本

回顾 2025 的系统选型，讲者给出一个很有洞察力的说法：过去几年大家主要在追求 benchmark 更高；但当很多 use case 已经“够用了”，接下来的竞争会越来越落在第二条 Pareto frontier 上，也就是“在保持足够高质量的同时，把推理成本、延迟和能耗继续压低”。

这也是 small language models、routing、更细的 serving optimization 会持续升温的原因。未来的研究重点不只是“能不能做”，更是“能不能便宜地做、稳定地做、大规模地做”。

4.3 硬件重新进入叙事中心

Hardware optimization



Stanford University

Figures adapted from "Analog in-memory computing attention mechanism for fast and energy-efficient large language models", Leroux et al., 2025.

ICME

图 5: 课程中的硬件优化示意: 不是仅仅改软件 kernel, 而是尝试把 attention 所需的一部分算子与数据流更深地编码进硬件。

课程在这里把话题从模型算法拉回到硬件现实。GPU 擅长矩阵乘法, 但 Transformer 的注意力并不只需要矩阵乘法, 它还高度依赖 KV 读写、内存搬运和中间状态管理。FlashAttention 一类工作已经说明, 很多瓶颈来自 memory movement, 而不是算子本身。

更进一步, 讲者介绍了一类模拟硬件方向的工作, 尝试把某些 attention 相关计算直接体现在电路和信号流里。无论这条路最终是否成为主流, 它都说明一个事实: 大模型系统性能的上限, 并不只由算法决定, 也受制于底层计算介质。

4.4 本章小结

这一节想表达的不是“某个具体热点会胜出”, 而是三种长期张力: 跨模态借鉴会继续、效率竞争会增强、硬件与系统实现会重新变得像模型结构一样重要。

5 Closing Thoughts: 真实世界会如何消化这些能力

5.1 日常工作与学习已经被重塑

课程的 closing thoughts 不是抽象口号, 而是从学生和工程师的日常体验出发。讲者认为, coding assistants、浏览器级 AI、文案与创意草稿、课堂中即时用 ChatGPT 澄清概念, 这些都已经是现实存在的价值场景。尤其对学习而言, 得到一个“即时讨论对象”和“低成本反馈回路”本身就是很大的增益。

这段话与整门课前面的内容其实是一致的: 真正有价值的系统未必是最炫的 benchmark 冠军, 而是能把知识获取、执行效率和认知负担降下来。

5.2 Agent 的民主化与 OS-level 想象

More use cases to be expected

Tomorrow.

Democratization of existing agents (e.g. Google Workspace launch)

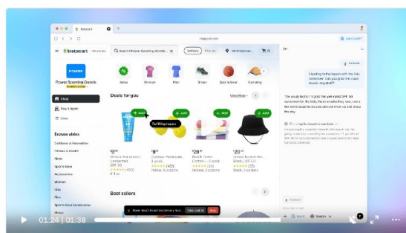
Near term.

Browser? (e.g. OpenAI's Atlas launch). Looking further: OS-level LLM?

Introducing ChatGPT Atlas

The browser with ChatGPT built in.

Download for macOS



Stanford University "Introducing ChatGPT Atlas", OpenAI, October 21, 2025.

ICME

图 6: 课程以 Atlas 等产品作为“近未来”信号: agentic workflow 可能从专家工具走向更广泛用户界面, 先浏览器化, 再逐步操作系统化。

讲者判断, 未来几年一个重要方向是 agent 能力的 democratization。也就是说, 今天只有熟悉该领域的人会用的 agentic workflow, 会逐渐通过自然语言界面进入普通人的日常工作流。浏览器级助手、桌面级助手、移动端 OS-level assistant, 都属于这条演化链的一部分。

但课程同样保留了现实主义: agent 步数越多, 失败概率越高; 安全、权限、注入攻击、防止越权执行, 仍然是阻碍大规模普及的主要因素。讲者甚至把“你是否愿意真的让 AI 客服替代人工客服”当作一个现实测试, 借此说明 empathy、groundedness 和对隐性规则的把握, 仍然是困难问题。

5.3 仍未解决的长期难题

最后一讲没有用“AGI 快到了”之类的句子收尾, 而是回到几个更朴素但更硬的问题:

1. 固定权重问题: 模型训练完成后权重冻结, RAG 和 tools 只是绕开了连续学习问题, 并没有真正解决它;
2. Hallucination 问题: 从 next-token prediction 目标出发, 模型天生不是“事实数据库”;
3. Personalization、interpretability、safety: 这些都还远未被彻底解决。

讲者给学生的最后建议则非常具体: 想持续跟进该领域, 就去看 arXiv、NeurIPS / ICML / ICLR / ACL / EMNLP 等 venue; 不要只读论文, 也要读作者给出的代码; 此外, Hugging Face 的 trending papers 这类入口, 可以帮助你快速捕捉近期热点。

课程结尾的现实判断

未来突破未必只来自更大的模型或更高的 benchmark。它同样可能来自更好的数据、更便宜的推理、更合适的硬件、更新的体系结构, 或者更贴近真实任务的产品接口。

5.4 本章小结

Closing thoughts 把技术趋势重新落回真实使用场景：LLM 已经开始深刻改变学习、编码与信息获取方式，但要走向普遍可依赖的 agent 系统，还需要解决稳定性、安全性和连续学习等长期问题。

6 总结与延伸

第 9 讲并没有试图预测某一条唯一正确的未来路线，而是用回顾与展望告诉我们：大模型领域仍然处在快速重构阶段。课程至少给出了四个值得长期记住的判断。

1. Transformer 很重要，但未必是最终形态；它已经扩展到视觉、多模态，也可能被新的结构部分替代。
2. 自回归 LLM 依然是今天的主流，但 diffusion、并行生成等路线说明“逐 token 串行生成”并非唯一答案。
3. 未来竞争会越来越看重质量与成本的联合最优，而不是单维 benchmark。
4. 真正决定产品形态的，不只是模型分数，还包括硬件、系统接口、安全边界与用户 workflow。

如果把整门课的收获压成一句话，可以说：**理解大模型，不只是理解一个网络结构，而是理解一个不断在数据、训练、推理、外部工具、评测体系和真实世界约束之间重新平衡的系统。**

对已经学完这门课的人来说，最有价值的下一步不是死记热点，而是练习两种能力：第一，能把任何新论文放回“它在解决哪一类老问题”的坐标系里；第二，能把任何新模型放回“它是否真的改善了我的任务、成本和风险”的工程坐标系里。

这也正是本课程最后一讲的意义所在：它不只是总结学期内容，而是在训练一种继续学习这门领域的方法。